

A deep residual reinforcement learning algorithm based on Soft Actor-Critic for autonomous navigation

Shuhuan Wen^{a,b}, Yili Shu^{a,b}, Ahmad Rad^{c,*}, Zeteng Wen^{a,b}, Zhengzheng Guo^{a,b}, Simeng Gong^{a,b}

^a Engineering Research Center of the Ministry of Education for Intelligent Control System and Intelligent Equipment, Yanshan University, Hebei Street, Qinhuangdao 066004, Hebei, China

^b Key Lab of Industrial Computer Control Engineering of Hebei Province, Yanshan University, Hebei Street, Qinhuangdao 066004, Hebei, China

^c School of Mechatronic Systems Engineering, Simon Fraser University, 250-13450, 102 Avenue, Surrey, BC V3T 0A3, Canada

ARTICLE INFO

Keywords:

Path planning
Deep residual reinforcement learning
Prioritized experience Replay
Curiosity-driven

ABSTRACT

The problem of autonomous navigation has attracted significant attention from robotics research community in the last few decades. In this paper, we address the problem of low data utilization due to the large amount of episode experience value data. A maximum entropy algorithm based on prioritized experience replay (Learning Good Experience based on Soft Actor-Critic, LGE-SAC) is proposed to quickly reproduce past good experience episodes. As the deep reinforcement learning method is susceptible to failure to plan ahead and explore the target position in a long sequence environment, a deep Residual Soft Actor-Critic (RSAC) is proposed to alleviate this problem. The reinforcement learning policy is fused with the Artificial Potential Field method to improve the generalization ability of the proposed algorithm, thus improving robot adaptation in new test environments. In order to validate the effectiveness of the proposed algorithm, we conducted simulation experiments in Gazebo simulator environment and real experiments on a Turtlebot3 robot equipped with LiDAR sensor. Simulation and experiment results show that the proposed algorithm effectively avoids obstacles and succeeds in reaching the goal compared to other obstacle avoidance algorithms. In comparison with the Artificial Potential Field method, the planning success rate of the proposed RSAC algorithm in the test environment is increased by 30%, and at the same time, the number of planning steps is reduced by half, and the generalization ability is improved.

1. Introduction

Robotics research has benefited from the rapid development of artificial intelligence technology which has been employed in almost all areas of robotics navigation (Lashkari, Biglarbegian, & Yang, 2020; Truong & Ngo, 2017; Xiao, Liu, Warnell, & Stone, 2017). The autonomous navigation ability of a mobile robot is an essential attribute that is required in various tasks. The robot has to complete its tasks in complex and diverse environments while learning to avoid obstacles, which is one of the sub-problems that need to be resolved (Ejaz, Tang, & Lu, 2021; Watkins-Valls, Xu, Waytowich, & Allen, 2020; Wortsman, Ehsani, Rastegari, Farhadi, & Mottaghi, 2019).

Most of the traditional path planning methods rely on a priori map, such as A* algorithm (Zhang Zhe, Wu, Dai, & He, 2020), and Dijkstra algorithm (Lotfi et al., 2021). As such, the performance of the

mentioned methods considerably deteriorates in unknown settings. The RRT algorithm (Ye, Dong, Zhao, & Huang, 2022) explores unknown areas by randomly expanding the nodes of the tree, but it is prone to the problem of entrapment in local optima or failing to generate an optimal path. Traditional methods possess several noteworthy limitations, including low efficiency, poor real-time performance, and susceptibility to falling into local optima. Furthermore, in dynamic environments, these methods cannot guarantee a satisfactory performance and may even fail to search for a path (Yang, Ni, Xi, Wen, & Li, 2023). Moreover, these are prone to falling into local minima.

In the context of path planning, particularly in unknown environments, reinforcement learning algorithms are often regarded as more appropriate and effective (Wu et al., 2023). Compared with conventional methods, an autonomous robot enabled with the Reinforcement Learning (RL) learns via “trial and error” and is rewarded and guided by

* Corresponding author.

E-mail addresses: swen@ysu.edu.cn (S. Wen), yli@stumail.ysu.edu.cn (Y. Shu), arad@sfu.ca (A. Rad), wenzeteng@cmbc.com.cn (Z. Wen), gzz@stumail.ysu.edu.cn (Z. Guo), gsm@stumail.ysu.edu.cn (S. Gong).

<https://doi.org/10.1016/j.eswa.2024.125238>

Received 26 February 2024; Received in revised form 19 July 2024; Accepted 26 August 2024

Available online 30 August 2024

0957-4174/© 2024 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

interacting with the environment (Chalmers, Contreras, Robertson, Luczak, & Gruber, 2018; Xie, Li, Zhang, & Zhu, 2019; Zhang, Zhen, Wang, & Gao, 2021).

In an autonomous navigation strategy, the robot observes the current state through its on-board sensors; and the observed content constitutes the necessary features for decision-making. Based on the obstacle avoidance strategy, the robot outputs the corresponding actions and executes them to reach at a new state. Each interaction between the robot and the environment results in a reward value. If the robot executes the correct action as planned, the reward value is positive; if the robot executes an incorrect action, such as a collision, it receives a negative reward. The purpose of RL is to enable the agent to achieve the largest cumulative discounted reward.

The RL state space is generally high-dimensional and continuous. The essence of the conventional reinforcement learning approach, Q-learning, resides in assessing the long-term worth of executing a particular action within a defined state via the utilization of the Q function. Specifically, the Q-function quantifies the expected total future reward achievable by executing action in state. In Q-learning, Q-values are stored in a table, where each state-action pair corresponds to a Q-value, guiding the decision-making process of the agent. However, this approach becomes impractical in high-dimensional and continuous state spaces, as the required size of the Q-table grows exponentially with the increase in the dimensionality of the state space.

To address the aforementioned constraint, deep learning (DL) has been integrated with RL, which is deep reinforcement learning (DRL). DRL can approximate the Q-function well due to its powerful capabilities of neural networks, and perform effective learning and optimization of strategies in high-dimensional state spaces. For example, Soft Actor-Critic (SAC) aims to maximize cumulative rewards while balancing exploration and exploitation through entropy regularization. In SAC, the learning process of the Q-function is accomplished by minimizing the mean squared error loss, ensuring that the predicted values of the Q-function gradually approximate their true values, thereby guiding the policy network towards optimal updates.

In recent years, DRL has been employed for robot navigation (Pfeiffer et al., 2018; Rana, Dasagi, Talbot, Milford, & S nderhauf, 2020; Shi et al., 2020; Ye et al., 2020). The environmental features are extracted as robot states and the robot takes action according to a policy of obstacle avoidance. By continuously optimizing and updating the policy, the robot can successfully avoid obstacles to reach the target point.

Although the DRL method solves the high-dimensional input problem, it usually receives sparse rewards in navigation and requires a large number of training sets to converge. Most of the existing DRL algorithms demand a large amount of data to learn a good policy, and cannot be temporarily applied to high data sampling costs or long-term situations. In order to improve the efficiency of the used samples, we need appropriately collect valuable information of the environment, and obtain as many rewards as possible. At present, most DRL methods for navigation mainly focus on improving the exploration efficiency by designing better strategies (Li et al., 2020; Yoon et al., 2020). Pathak et al. (2017) proposed an Intrinsic Curiosity Model (ICM) to alleviate the sparse reward problem and the robot exploration stuck problem through the intrinsic reward model.

In a long sequence navigation environment, the space to be explored by robots increases and it is difficult to reach the goal. The above methods ignore the use of experience. At the same time, when the robot enters an unknown environment, it is difficult to plan a collision-free path by an existing overfitting model (Frans & Isola, 2023; Malik, Li, & Ravikumar, 2021; Moon, Lee, & Song, 2022). DRL is encouraged to learn some high dimension features to improve the generalization under unknown environments (Lu, Lee, Abbeel, & Tiomkin, 2020).

In order to successfully reach the goal without colliding with obstacles in new dynamic and static environments, the robot needs to have the ability to quickly adapt to the new environment. In this paper, we present a new path planning algorithm based on deep residual

reinforcement learning that addresses the above issues. The main contributions of the paper are as follows:

- 1) Aiming at the problem of low data utilization, a novel Learning Good Experience based on Soft Actor-Critic (LGE-SAC) is proposed. It is a new prioritized experience replay method based on the difference between the rewards in an episode and the estimated value of the value function. Compared with Prioritized Experience Replay (PER) method, LGE-SAC does not need importance sampling, which makes up for the defect that if there is a big gap between the past experience and the present, it will have little effect. In the policy update process, the LGE-SAC algorithm identifies state-action pairs where the reward obtained in an episode exceeds the estimated value of the value function, triggering a secondary update. By leveraging past successful experiences, the LGE-SAC algorithm enhances both the speed of policy convergence and the overall reward.
- 2) We combine attention mechanism and ICM module to focus the robot on beneficial state information. The combination of intrinsic and extrinsic rewards enhances exploration in long sequences, especially in original reward-sparse environments where it mitigates planning failures.
- 3) We propose a deep residual reinforcement learning based on SAC framework that uses the Artificial Potential Field method as a prior controller and outputs hybrid actions in combination with reinforcement learning methods to correct prior knowledge. In the unknown environment, the decision to take the output action of the prior controller or the hybrid action is made by discriminating the uncertainty to improve the generalization performance in the unknown environment.

The rest of this paper is organized as follows. Section II mainly introduces the navigation method based on deep reinforcement learning and the related work of episodic control. Section III shows the framework of our navigation system. Section IV presents our proposed algorithm, including LGE-SAC algorithm and Residual Soft Actor-Critic (RSAC) algorithm, and evaluates the simulation results. The paper is concluded in Section V.

2. Related work

Most related studies have focused on the improvement of navigation ability (Wen, Wen, Zhang, Zhang, & Wang, 2021), including optimistic navigation (Fu, Co-Reyes, & Levine, 2017; Osband, Blundell, Pritzel, & Van Roy, 2016), posterior sampling, and information gain. Haarnoja et al. (2018) proposed SAC to maximize the entropy while maximizing the reward policy, which improved the agent's navigation ability and made the model effect more stable. The above papers focus on the research of policy navigation ability; however, they do not consider the accumulated experience of the agent. In this paper, we exploit this valuable information to further improve the navigation capabilities. In this section, we briefly review the residual reinforcement learning and deep reinforcement learning methods for navigation.

Residual Reinforcement Learning: The residual reinforcement learning framework is to learn a residual strategy to correct the prior by using a prior controller. The residual policy is the designed autonomous path planning strategy based on reinforcement learning. Since the input part adds the action vector output by the prior controller to improve the stability of the planning, it is referred to as the residual policy. The output is generated by summing the output of the prior controller and the learned strategy.

Rana, Talbot, Dasagi, Milford, and S nderhauf (2020) combined the TD3 algorithm (Bach et al., 2020) with the Artificial Potential Field method to solve the problem of poor generalization performance of robots in long sequence environments. Rana, Dasagi, Talbot, Milford and S nderhauf (2020) proposed a multi-controller fusion method where the actions of a priori controller and the output of the RL actions

are not simply summed, but reconstruct the hybrid Gaussian distribution of both. Rana et al. (2023) elaborated and verified the feasibility of (Rana, Dasagi, Talbot, Milford, & Sünderhauf, 2020) using Bayesian theory and implemented planning in mobile robot navigation and robotic arms. Iscen et al. (2018) proposed to learn the residual policy while also modifying the parameters of the classical controller. However, directly modifying a priori controller using the policy may not guarantee that the action output is reliable. Although the solution provided by a priori controller is non-optimal in some cases, it is able to guarantee the safety of the agent. Xia et al. (2020) introduced a framework that combines a learning strategy for predicting sub-goals with a motion planner and trajectory executor required to achieve these sub-goals, enabling adaptive robot navigation in various environments. Carroll et al. (2023) developed a kernel-based residual learning framework. Initially, a kernel neural network was trained using data collected from a model predictive controller. In addition to the frozen kernel network, reinforcement learning was employed to train the residual controller network to achieve excellent performance and robustness against external disturbances. Ahmed et al. (2022) explored the use of convolutional-residual block mixtures in end-to-end goal-directed robot navigation, focusing on learning strategies for residual blocks in deep/large convolutional architectures under unknown scenarios. Xue and Chen (2024) integrated classical motion planning algorithms with DRL algorithms. The upper-level DRL algorithm received raw sensor information to generate local goals, while the lower-level classical planner generated smooth and safe trajectories to reach the goals. The framework primarily relies on the classical motion planner for obstacle avoidance, but it only considers static obstacles. Overall, residual learning is often employed to enhance stability and generalization.

Deep reinforcement learning: Thanks to the advances in high-performance computing hardware, deep neural networks have shown great potential in solving complex estimation problems. The deep reinforcement learning has succeeded in many tasks, including Atari games and robot control (Henriques & Vedaldi, 2018). Compared with traditional methods that need to model the environment and process sensor data, deep reinforcement learning methods establish an end-to-end framework and generate decisions that are similar to those that humans make.

Lin et al. (2018) combined Deep Q-Network (DQN) with episodic control, which solved the poor generalization of episodic control and made up for the slow convergence speed of DQN. However, it is only suitable for near-deterministic environments. Zhu et al. (2017) proposed a goal-driven visual navigation method. The currently observed RGB image and the target RGB image are input into the DRL framework to obtain action instructions. It realizes robot navigation in a discrete three-dimensional environment. Fang et al. (2019) proposed storing all past embedding states and use the attention mechanism to encode the historical trajectory as a decision input, which can improve data utilization and be suitable for long-sequence navigation. Du, Yu, and Zheng (2020) introduced a trial-driven imitation strategy that used the Dijkstra algorithm to generate prior expert knowledge to guide the learning of agents. However, their control commands are only simple discrete actions, such as turning left, turning right, and going straight, which only leads to rough navigation behavior. Ma et al. (2021) expanded the traditional behavioral cloning approach using priority sampling techniques and integrated it with reinforcement learning algorithms, yet they assumed the availability of a small number of expert demonstrations as references. Huang, Zhao, Yue, and Wang (2024) proposed a two-level planning framework that unified DRL with linear programming; however, this approach is tailored for discrete action spaces. Liu et al. (2023) introduced attention and hierarchical mechanisms to address issues such as unstable training, slow convergence, and excessively large state spaces in autonomous navigation, presenting a highly effective DRL framework for autonomous navigation and obstacle avoidance. Nevertheless, this method did not address the generalization to unknown environments and dynamic obstacle avoidance.

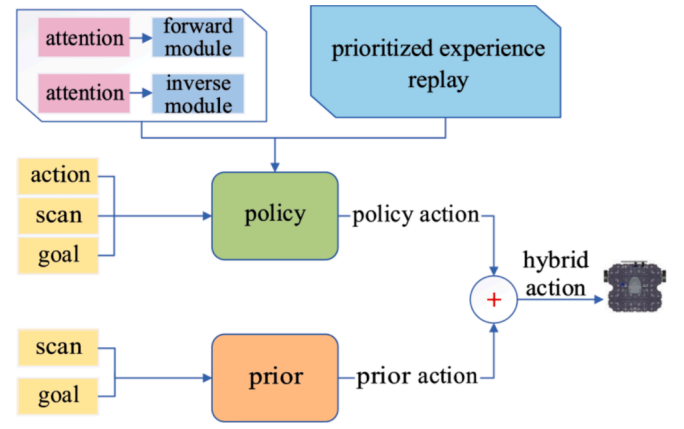


Fig. 1. The system framework of the robot based on RSAC algorithm.

In general, purely enhancing the exploration ability of the policy may cause unnecessary actions, which are not conducive to policy convergence, and over-exploitation may cause the policy to fall into a local optimum. Therefore, in order to address low data utilization and its generalization performance, we propose a novel prioritized experience replay method to rapidly reproduce good experience episodes and improve data utilization. An attention-based curiosity-driven mechanism is incorporated to improve the navigation performance of the robot. The Artificial Potential Field (APF) method is used as a prior controller to improve the algorithm's generalization ability in unknown environments.

3. Methodology

In this section, we systematically introduce our methodology. Firstly, we provide an overview of the overall framework of the navigation system, laying a solid foundation for subsequent in-depth discussions. Subsequently, we delve into the application of reinforcement learning in navigation systems and elaborate on the implementation details of the proposed LGE-SAC algorithm and RSAC algorithm. These algorithms aim to address navigation challenges in complex environments, enhancing the autonomy and adaptability of the system.

3.1. The framework of the proposed navigation system

The proposed overall navigation framework is shown in Fig. 1. When the robot explores the environment, we use the information obtained by Light Detection and Ranging (LiDAR) (Lai, Chen, Lu, Liu, & Jia, 2023) as

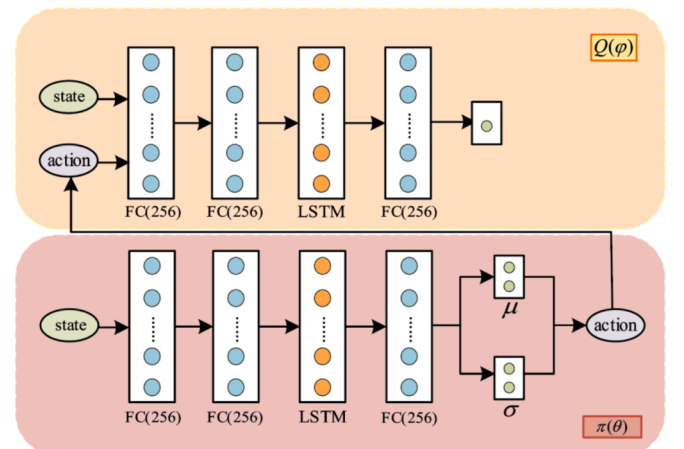


Fig. 2. Network structure.

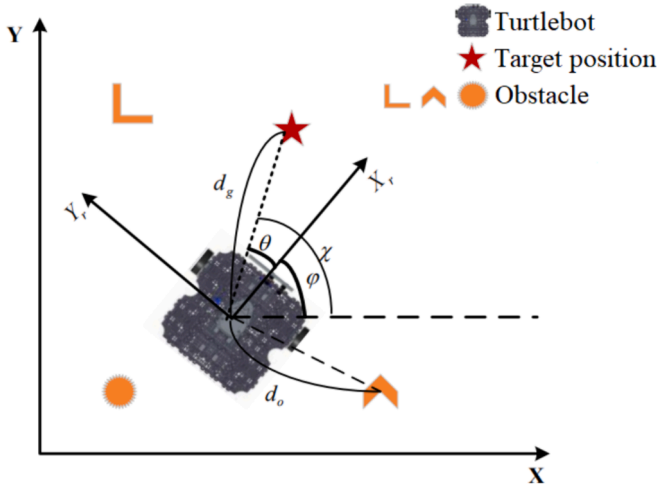


Fig. 3. Robot State Space Model.

input, and output reasonable motion information (including the robot's linear velocity and angular velocity). The complete framework includes two parts, a reinforcement learning policy that incorporates prioritized experience replay and a prior controller based on Artificial Potential Field method. The proposed framework integrates the effects of both output actions on the robot. The update of the policy is performed while the interaction with the environment is repeated. In the reinforcement learning policy part, we use a SAC-based algorithm that introduces a novel prior experience replay approach to quickly reproduce good experience episodes. Moreover, an attention-based intrinsic reward is added to further improve the exploration performance of the robot. The navigation strategy is constantly updated until it converges. Finally, the robot can successfully avoid obstacles and reach the target position.

In the policy update part, we adopt an end-to-end deep reinforcement learning framework. The overall neural network structure is shown in Fig. 2 and consists of four layers. The first two layers are Fully Connected (FC). In order to retain time series information, the third layer introduces the Long Short-Term Memory (LSTM) layer (Hochreiter & Schmidhuber, 1997), and the fourth layer is a fully connected layer. The critic network outputs the Q function, and the actor network outputs the μ and variance σ , constructing Gaussian distribution and re-sampling to obtain the linear velocity and angular velocity of the robot.

3.2. Deep reinforcement learning

The obstacle avoidance process is regarded as a decision-making process. In this paper, the robot interacts with the environment through LiDAR and continuously improves its action prediction policy. The DRL elements are as follows:

State-space representation: Assuming the robot is in the environment shown in Fig. 3. Among them, $X-Y$ is the world coordinate system of the navigation environment, and X_r-Y_r is the relative position coordinate system of the robot itself during the motion process. Set the robot's forward direction to the x axis and the counterclockwise 90° orientation to the y axis. Assuming that in the $X-Y$ coordinate system, the robot's pose at time t is (x_t, y_t, φ) , that is, the coordinates are (x_t, y_t) , and the heading angle is φ . χ is the angle between the target position and the horizontal direction of the $X-Y$ coordinate system. Since φ and χ can be obtained from the known information, the angle between the robot's orientation and the target position can be calculated: $\theta = \chi - \varphi$.

The robot continuously obtains environmental information, that is, the distance from obstacles, through the onboard two-dimensional LiDAR. The state space is mainly composed of five components, including: two-dimensional LiDAR data (including the distance d_o between the robot and the obstacle) pose information of the robot at the

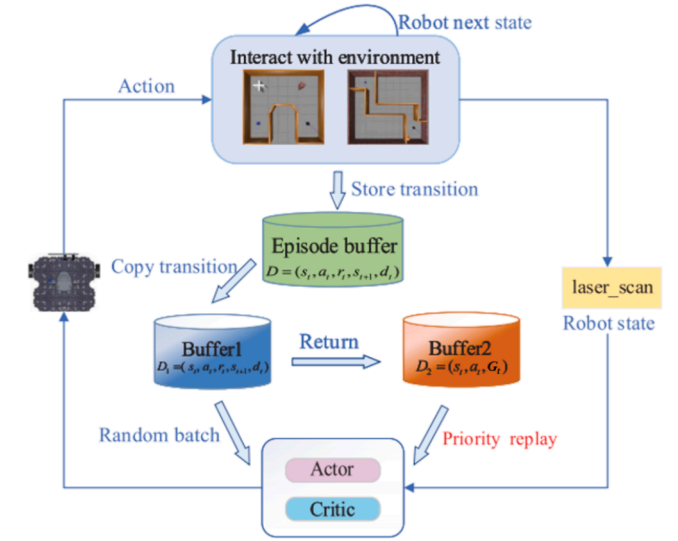


Fig. 4. The framework of LGE-SAC algorithm.

moment (x_t, y_t) , the distance d_g between the robot and the target position, the robot's actions (including angular velocity ω_t and linear velocity v_t), the angle θ formed by the line and the horizontal position of the obstacle.

Design of reward function: The reward function is an essential factor that affects whether the reinforcement learning algorithm can successfully converge. If the reward function is unreasonably designed, the robot will not explore the target position or fall into a local optimum. The reward function in this paper is designed as follows:

$$r_t = \begin{cases} r_{arrive}; & \text{if } d_g^t < d_a \\ r_{collision}; & \text{if } o_{min} < 0.46 \\ (d_g^t - d_g^{t-1}) \times r_{arrive} + p; & \text{otherwise} \end{cases} \quad (1)$$

where d_g^t is the distance between the robot and the target position at the moment, and d_g^{t-1} is the distance between the robot and the target position at the last moment. d_a is the threshold to determine whether the robot has reached the target position. When $d_g^t < d_a$, the system judges that the robot reaches the target position and gets a positive reward value r_{arrive} , we set it to 100. o_{min} is the shortest distance in the laser data, that is, when the shortest distance is less than 0.46, the system determines that the robot has collided and gets a negative reward value $r_{collision}$, we set it to -50. In the process of walking normally, when the target location is not reached or there is no collision, if the robot is farther from the target position than the previous moment, we will give the robot a negative reward value. If the robot is getting closer and closer to the target position, give it a positive reward value. And at every moment, we give the robot a time penalty p to drive it to explore a shorter path and reach the target position faster. We set the time penalty to -0.05.

3.3. Learning good experiences method based on Soft Actor-Critic algorithm

We need to develop a suitable obstacle avoidance policy to complete the collision-free navigation task. The framework of the algorithm is shown in Fig. 4.

Both the actor and critic network update consist of two parts, as shown in (2).

$$\begin{aligned} J_{actor} &= J_{actor}^{SAC} + J_{actor}^{LGE} \\ J_{critic} &= J_{critic}^{SAC} + J_{critic}^{LGE} \end{aligned} \quad (2)$$

where J_{actor}^{SAC} and J_{actor}^{LGE} respectively represent the loss function of the actor network in different update stages, J_{critic}^{SAC} and J_{critic}^{LGE} respectively represent the loss function of the critic network.

In the first update stage, we use the SAC algorithm to optimize the policy. The agent gets a reward proportional to the entropy of the policy corresponding to the current time step. In this way, the objective function of the optimal policy (Haarnoja et al., 2018) is:

$$J_{\pi} = \sum_{t=0}^{T-1} \mathbb{E}_{(s_t, a_t)} [r(s_t, a_t) + \alpha H(\pi(\bullet | s_t))] \# \quad (3)$$

where α is the entropy regularization coefficient, that determines the relative importance of the entropy term against the reward. $H(\pi(\bullet | s_t))$ represents the entropy of the policy at each time step, and the specific calculation method is shown in formula (4) (Haarnoja et al., 2018).

$$H(\pi(\bullet | s_t)) = E(-\log(\pi(\bullet | s_t))) \# \quad (4)$$

It represents the randomness of the probability distribution corresponding to the stochastic policy ($\pi(\bullet | s_t)$), and the higher the randomness of the policy, the greater the entropy value.

In SAC, the learning of the Q-function in the critic network is achieved by minimizing the mean square error loss function (Haarnoja et al., 2018):

$$J_{critic}^{SAC} = \mathbb{E}_{(s_t, a_t)} D_1 \left[\frac{1}{2} (Q_{\theta}(s_t, a_t) - Q(s_t, a_t))^2 \right] \# \quad (5)$$

where D_1 represents the distribution of previously sampled states-action pairs.

The objective function of SAC policy is to minimize the Kullback-Leibler (KL) divergence between the two distributions. Therefore, the policy parameters can be learned in following equation (Haarnoja et al., 2018):

$$J_{actor}^{SAC} = D_{KL}(\pi_{\phi}(\cdot | s_t) \| \exp(Q_{\theta}(s_t, \cdot) - \log Z_{\theta}(s_t))) \# \quad (6)$$

where Z_{θ} normalizes the policy distribution. Although it is usually difficult to process, it does not contribute to the gradient of the new policy, so we can ignore it in the process of updating the policy.

In the second update phase, to improve the utilization of episode experience, we do not adopt the method of randomly sampling from the experience pool. Still, when the episode cumulative reward value $G_t = \sum_{i=t}^{\infty} \gamma^{i-t} r_i$ is greater than the state value function, the policy network and value function network are updated in the direction of excellent episode experience. Since the Temporal Difference method (TD) considers the difference in reward between adjacent states, and the value function can be updated at each step without waiting for the end of the entire episode of exploration. The Monte Carlo method (MC) (Oh, Guo, Singh, & Lee, 2018) considers the cumulative reward value of the entire episode, which requires a large amount of calculation. So in general, TD method converges faster than MC method. Although the variance of the TD method is small, the deviation is larger and the accuracy is reduced by comparing with the MC method. Therefore, in this paper, the MC method is used to calculate the sum of the benefits of each state, that is, after the data collection of each episode, the reward value obtained by each state in this episode up to the current state is calculated. We store it in an experience buffer, expressed as: $D_2 = (s_t, a_t, G_t)$, where s_t and a_t are state and action at time t . G_t is the discounted reward in future, where γ is a discount factor.

To use the good state-action pairs in the experience buffer, we use the detailed explanation in (Oh et al., 2018) to construct the loss function of the policy network and the critic network:

$$J_{actor}^{LGE} = \log \pi_{\phi}(a_t | s_t) (G_t - V^{\pi})_+ \# \quad (7)$$

$$J_{critic}^{LGE} = \frac{1}{2} \|(G_t - V^{\pi})_+\|^2 \# \quad (8)$$

where $(G_t - V^{\pi})_+$ means that when the episode cumulative reward value is greater than the value function estimate at this time, we update the critic network and the actor network. If the reward in the past is greater than the value estimated by the network, the policy tends to learn the past good state-action pairs with a large reward. On the contrary, when the reward is smaller than value function, the experiences are not used to learn good policy. Overall, this design encourages the agent to imitate a good experience transition, thereby obtaining a higher reward value. This idea is suitable for static environments in which the robot behaves almost deterministically (no slippage, uncontrolled movement, etc). J_{critic}^{LGE} updates the estimate value towards Monte-Carlo reward.

The vast amount of data in the experience buffer will cause the neural network to converge slowly by randomly extracting data for updating. In order to improve the utilization of data, we adopt the sampling method of prioritized experience replay instead of random mini-batch sampling, and use good experience trajectories to iteratively update the parameters of the policy and value function.

The difference from classic prior experience replay is that traditional prioritized experience replay (Fang et al., 2019) uses TD error to measure the priority of experience and involves importance sampling. We know that when G_t is greater than V^{π} , it is helpful to learn the good past experience, that is, it is beneficial to the gradient update. Therefore, in order to increase the effective sampling ratio, the sampling priority (Schaul, Quan, Antonoglou, & Silver, 2015) we set is based on the clipped advantage $(G_t - V^{\pi})_+$. In the stage of imitating good past experience, we sample from experience buffer D_2 .

In general, the learning of the optimal policy is divided into two stages. In the first stage, we perform off-policy learning using data $(s_t, a_t, r_t, s_{t+1}, d_t)$ sampled from the experience buffer, leveraging the SAC algorithm for initial policy optimization. Subsequently, the second stage is to sample transitions (s_t, a_t, G_t) from experience buffer D_2 to perform imitation learning on the good action states in the past. This constitutes a secondary update process, further enhancing the learning of effective strategies.

3.4. Residual Soft Actor-Critic algorithm

The RSAC algorithm extends upon the LGE-SAC algorithm by introducing an attention-based curiosity reward mechanism in Section 3.4.1 to enhance the robot's environmental exploration capabilities. Additionally, in Section 3.4.2, it integrates the Artificial Potential Field method as a prior controller to form a deep residual reinforcement learning framework that augments the robot's adaptability in unknown environments.

3.4.1. Attention-based curiosity-driven method

Despite the dense external reward mechanism employed in previous studies, the robot's adaptability is limited to the training environment due to considering the distance to the target position when setting the external rewards, which leads to poor generalization ability in unknown environments. To address this issue, based on the robot's rational utilization of good experience, we expand its exploration scope and avoid local optimal. In this subsection, we design an internal reward mechanism based on curiosity, incorporating an attention mechanism for state and action selection. Furthermore, we modify the loss function for curiosity-driven exploration to further enhance its exploration capabilities.

Intrinsic Curiosity Model is a system that estimates whether a state is novel or not and gives the corresponding intrinsic reward. It consists of three parts of neural network: forward model, inverse model and feature extraction.

We use the attention mechanism to process the input information and extract useful feature information, which ensures that the input feature space is to be focused on, thus improving the training efficiency. The forward and inverse model equations are shown below (Reizinger &

Szemenyei, 2020)

$$\hat{\phi}(s_{t+1}) = f(\text{att}(\phi(s_t, a_t)); \theta_f) \# \quad (9)$$

$$\hat{a}_t = g(\text{att}(\phi(s_t)), \text{att}(\phi(s_{t+1})); \theta_l) \# \quad (10)$$

The intrinsic reward is defined in terms of the gap between the actual next moment state and the next moment state obtained by the forward model. Meanwhile, the learning of the parameters of the forward model is updated iteratively by minimizing the prediction error.

$$L_{\text{forward}} = r_t^i = \frac{1}{2} \|\hat{\phi}(s_{t+1}) - \phi(s_{t+1})\|_2^2 \# \quad (11)$$

The update of the inverse model is performed by minimizing the error between the predicted and actual actions. Since the continuous action is used in this study, the loss function of the inverse model is expressed using the mean squared loss function as:

$$L_{\text{inverse}} = \frac{1}{2} \|\hat{a}_t - a_t\|_2^2 \# \quad (12)$$

The goal of the curiosity-driven exploration mechanism is to minimize the probability of occurrence of next-moment states that are not valuable for planning. In order to discriminate whether the next moment state predicted by the forward model is beneficial to curiosity, an attention mechanism is added to the loss function part of the forward model to distinguish it, so the loss function of the curiosity-driven mechanism becomes as follows (Reizinger & Szemenyei, 2020), where β is a constant:

$$L_{\text{acm}} = (1 - \beta)L_{\text{inverse}} + \beta \cdot \text{att}(L_{\text{forward}}) \# \quad (13)$$

Since intrinsic rewards are introduced, the reward function becomes the sum of extrinsic and intrinsic rewards, as shown in equations (14), where r_t^i is shown in equation (11) and r_t^e is shown in equation (1).

$$r_t = r_t^i + r_t^e \quad (14)$$

The intrinsic reward (coefficient 0.01) obtained by the curiosity module is related to the state space vector and remains relatively small, even in unknown or dynamic environments, not exceeding the extrinsic reward (coefficient 1.0) that is directly tied to achieving the goal and increases as the robot approaches it. As a result, even in densely obstacle-populated environments, the robot explores in a direction closer to the target, reducing deviations from the goal due to curiosity. This naturally guides the robot to strike a balance between exploration and goal achievement.

3.4.2. Residual Soft Actor-Critic algorithm

RSAC algorithm is based on the LGE-SAC algorithm, adding the attention mechanism driver in the section 3.4.1. The RSAC network structure is consistent with LGE-SAC. We employ a deep residual reinforcement learning approach to enhance the robot's adaptability in unknown environments. Deep residual reinforcement learning consists of two main components: a residual policy to be learned and a classical controller, also known as a prior. The action output of the robot is determined by both components together and is expressed by the equation (Rana, Talbot, Dasagi, Milford, & Sünderhauf, 2020).

$$a_t = \pi_{\theta}(s_t) + \pi_{\text{prior}}(s_t) \# \quad (15)$$

where π_{θ} denotes the autonomous planning policy learned by reinforcement learning and π_{prior} denotes the action output by the classical controller. The prior output is gradually corrected by training using reinforcement learning to output a reasonable and valid action. In our research, the Artificial Potential Field method is used as the classical controller to provide prior knowledge. The Artificial Potential Field method is essentially a feedback control strategy that is robust to control and sensing errors, so the entire deep residual reinforcement learning framework (Rana, Talbot, Dasagi, Milford, & Sünderhauf, 2020) is

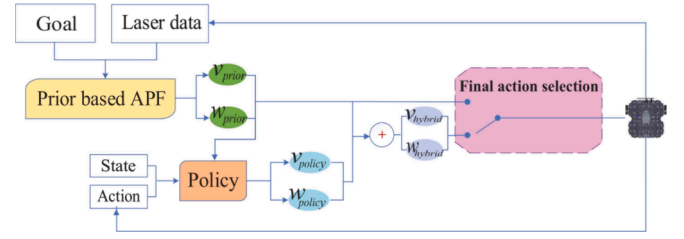


Fig. 5. Deep residual reinforcement learning framework.

shown in Fig. 5.

In the algorithm training process, the input of the prior controller part is the goal and the LiDAR scan data, in general, the output of the traditional path planning algorithm is the next moment position of the robot, in order to facilitate the combination with the action of policy learning, the prior controller also outputs a two-dimensional vector containing the linear and angular velocity $[v_{\text{prior}}, w_{\text{prior}}]$. The residual policy is the previously designed autonomous path planning strategy based on reinforcement learning. Since the input part adds the action vector output by the prior controller to improve the stability of the planning, it is called the residual policy. The output of the residual policy is denoted as $[v_{\text{policy}}, w_{\text{policy}}]$. The planning results of the two are then summed as the final action $[v_{\text{hybrid}}, w_{\text{hybrid}}]$ acting on the robot for the next moment of data collection.

Although prior knowledge is enhanced by learning the residuals, in cases where the residuals cannot be generalized, it may degrade the performance of the system and thus limit its reliability. To improve the generalization performance of the algorithm, we used a new action selection model during testing, which is the pink boxed switch section in Fig. 5. The uncertainty of the policy at this point is calculated to determine which method to use for action decisions. When the policy uncertainty is higher, the system is more inclined to choose the prior action output rather than the hybrid action. Although the behavior taken by a prior controller in some states may be suboptimal, it can try to ensure that the robot does not collide. We use the MC dropout method to measure policy uncertainty. The flowchart of RSAC algorithm is shown in Fig. 6.

When encountering trained or general situations, the behaviors learned from prior controllers and reinforcement learning can complement, adjust, and optimize each other. This strategy combines the advantages of prior knowledge and reinforcement learning, utilizing the stability of prior controllers and achieving adaptability to the environment through reinforcement learning. Although the solution provided by the prior controller may not be optimal in some cases, it can ensure the security of the agent. When encountering edge situations that are significantly different from the training data, the performance of reinforcement learning is often uncertain or unpredictable. This may lead to the model making incorrect decisions or producing inaccurate results in these situations. However, due to the addition of action vectors from the prior controller output in the input part of the residual strategy, the stability of the planning is improved.

4. Experiment and results

Regarding the experimental section, we first provide a detailed account of the experimental setup, the methods employed, and the evaluation metrics. Subsequently, we conduct comprehensive experiments to assess the performance of the LGE-SAC algorithm in both static and dynamic environments, and conduct an in-depth analysis of its performance characteristics. Furthermore, we also experimentally validate and analyze the RSAC algorithm. Lastly, through testing in real-world scenarios, we verify the feasibility and effectiveness of the proposed methods.

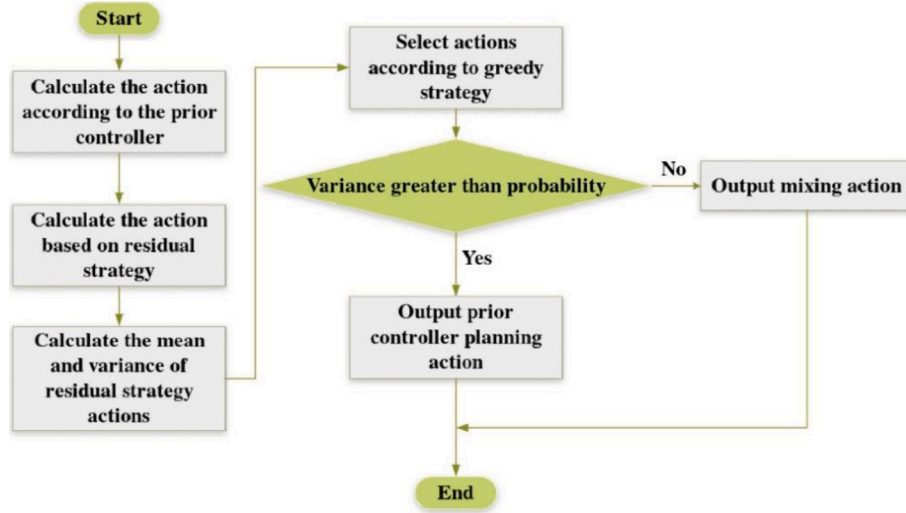


Fig. 6. RSAC algorithm testing process.

Table 1
Hyperparameters of algorithm.

Parameter	Value
Actor learning rate	0.001
Critic learning rate	0.001
Episode length	1000
Discount factor	0.99
Hidden units	256
Start steps	256
Intrinsic reward weight	0.0001

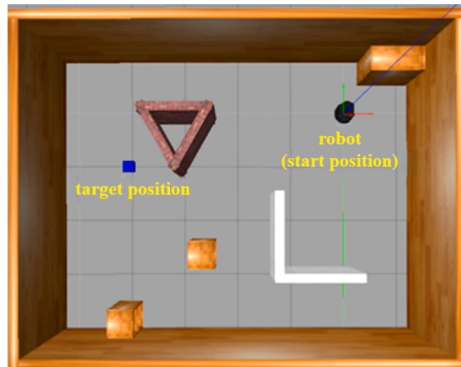
4.1. Experiment set up

In order to verify the effectiveness of the proposed LGE-SAC and RSAC, we conduct simulation experiments on the Gazebo simulation platform (Mingo et al., 2014) where we created an environment with obstacles of various shapes in the Gazebo simulator, and use turtlebot3 equipped with LiDAR and odometer as the robot model. In simulation experiments, we test our proposed algorithms in various environments, such as the environment with multi-shaped obstacles, irregular environment, corridor environment, dynamic environment and long sequence environment. In real experiments, we build two different experimental environments, including scene 1 with static obstacles and scene 2 with static and dynamic obstacles. We compare with other existing algorithms by some index values, such as Average reward, Robot speed, Success rate and high reward rate, Average instantaneous

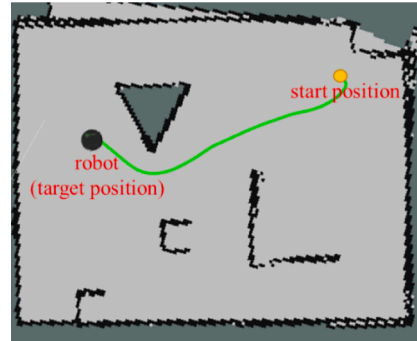
reward value of each episode and Planning steps of each episode. We complete the algorithm training process on a GTX1080Ti GPU. The neural network parameter settings during the training process are shown in Table 1.

In this section, we conduct comparative experiments of the following methods:

- 1) **SAC**: A classical reinforcement learning algorithm based on maximum entropy, on which our method is improved.
- 2) **Proximal Policy Optimization (PPO)**: PPO, a classical reinforcement learning method based on actor-critic, is applicable to both continuous and discrete action spaces, with actions optionally represented as two-dimensional vectors in continuous settings. Proposed by OpenAI to constrain policy updates and simplify calculations (Schulman, Wolski, Dhariwal, Radford, & Klimov., 2017), we compare it with our algorithm to assess convergence speed.
- 3) **Prior controller**: Artificial Potential Field, a traditional path planning method.
- 4) **PER-SAC**: An algorithm that combines traditional prioritized replay (Schaul et al., 2015) with SAC.
- 5) **LGE-SAC (ours)**: Our proposed approach based on prioritized replay of good experiences.
- 6) **RSAC (ours)**: Our proposed deep residual reinforcement learning method based on curiosity driven.



(a) Training environment



(b) Trajectory in the training environment

Fig. 7. Training environment with multi-shaped obstacles in Gazebo.

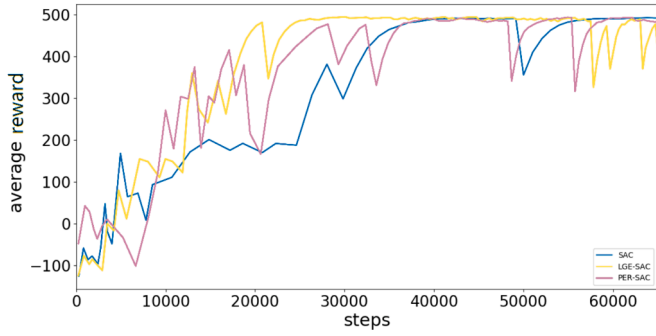


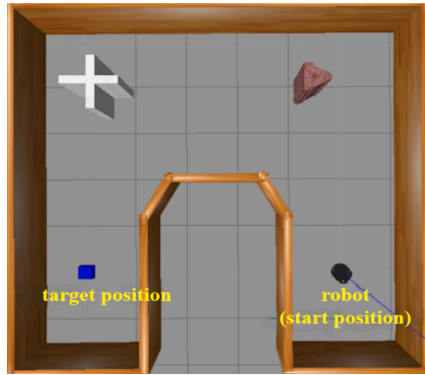
Fig. 8. Average reward curve for every 10 episodes of the three algorithms.

We first train the proposed model until it converges in a certain environment. Subsequently, we evaluated its planning effectiveness directly in the testing environment. Another way is to conduct further training in a new environment after the initial training, and finally evaluate its performance in a testing environment. Through this way, intelligent agents gradually adapt and learn to effectively plan in different environments.

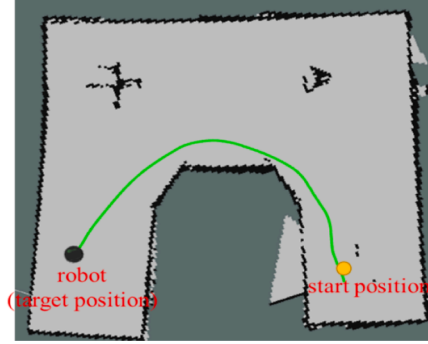
4.2. LGE-SAC performance

We train Turtlebot3 in a variety of environments, and the more typical environment is shown in Fig. 7(a). The size of this environment is 5 m*6m. The environment contains obstacles of various shapes, such as a cross, triangle, cuboid, etc. The blue square is the target point. During the robot navigation, if it collides with obstacles and surrounding walls, end this episode of exploration and start a new episode of exploration until the robot can reach the target position without collision. We compare the LGE-SAC algorithm proposed in this paper with PER-SAC algorithm with prioritized experience replay (based on TD error).

Due to the inconsistency in the number of exploration steps per episode, to present a clearer representation of the change in reward values, an average reward curve is plotted. Specifically, as the number of iteration steps increases, the average cumulative reward is calculated every 10 episodes in Fig. 8. The fluctuating phenomenon observed in the figure is caused by the algorithm getting trapped in an unstable local minimum. Specifically, the algorithm may have fallen into an unstable region of local optima, where slight variations in parameters or environmental disturbances can lead to fluctuations in the algorithm's performance. Although the average reward value of the PER-SAC algorithm in early exploration is higher than that of the SAC algorithm, the time it explores the target location is later than that of the LGE-SAC algorithm. It can be seen that the algorithm proposed in this paper can learn from good past experience and improve the utilization of experience.



(a) Irregular environment



(b) Trajectory in the irregular environment

Fig. 9. Irregular test environment in Gazebo and trajectory in Rviz.

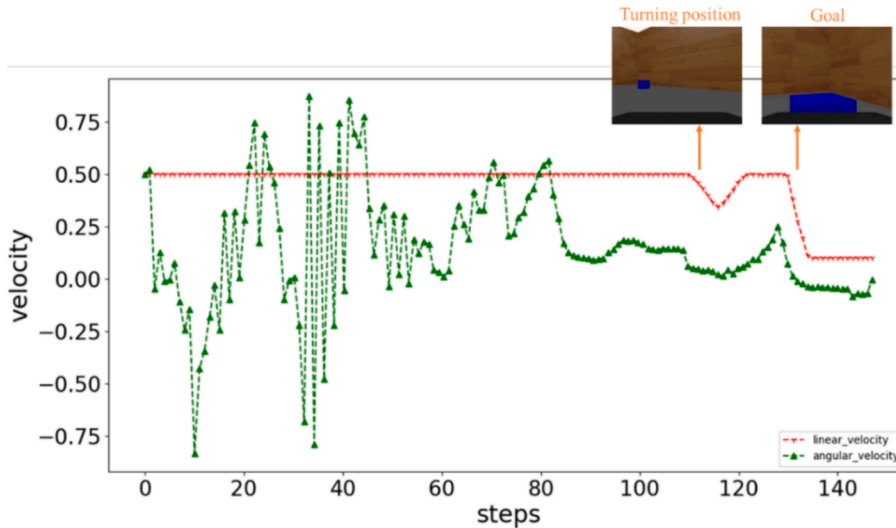


Fig. 10. Robot speed curve in irregular test environment.

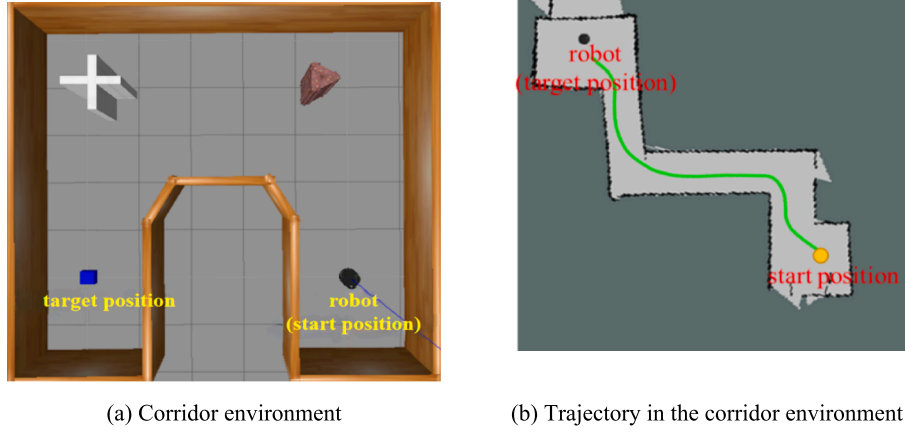


Fig. 11. Corridor test environment in Gazebo and trajectory in Rviz.

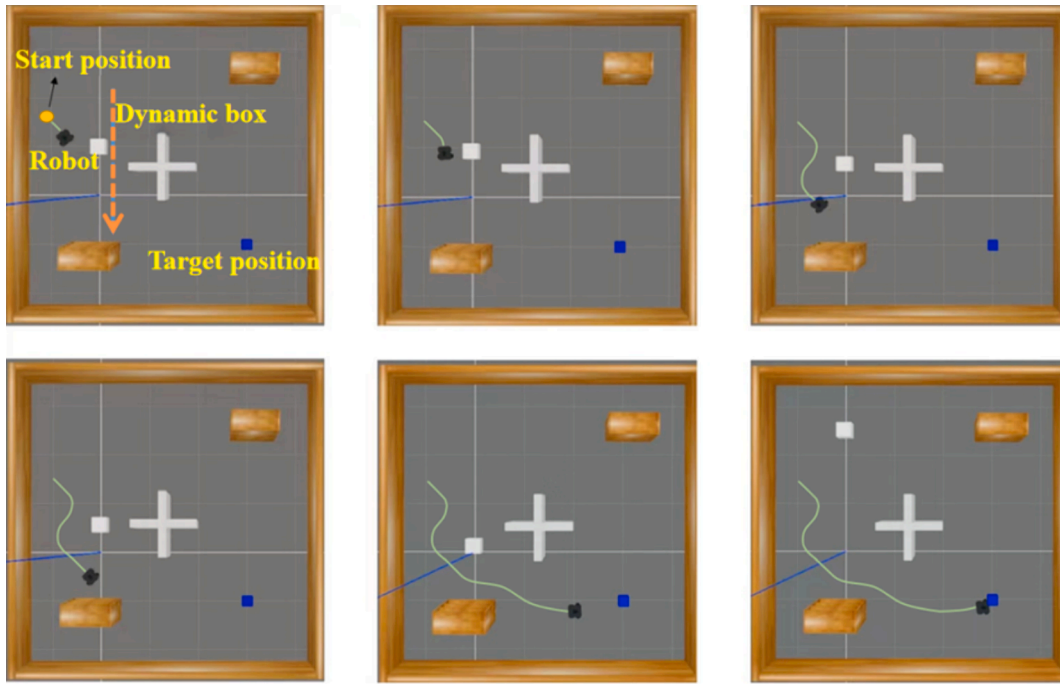


Fig. 12. Dynamic test environment in Gazebo.

For the purpose of testing the performance of the trained model, we will test the LGE-SAC algorithm in an unknown environment Fig. 9(a). Fig. 9(b) shows the irregular environment and the navigation trajectory of the robot. The size of this environment is 6 m*6m. The blue object in the left picture is the target point, the yellow circle in the right picture is the starting point, and the solid green line is the path diagram of the robot. We can see that the robot reaches the target position without collision in this test environment.

Fig. 10 shows the speed change curve of the robot. In the process of navigation, the robot can generally maintain the maximum linear speed (0.5 m/s) and reduce the speed when turning to avoid collision with the wall. When the robot safely passes the turn, it will resume its maximum speed and continue driving. When the target position is about to reach, the speed is significantly reduced to prevent missing the target position.

In order to test the validity of the proposed algorithm in the environments with obstacles of various shapes, we test the autonomous navigation ability of the robot in a narrow corridor environment with long sequence characteristics in Fig. 11. The size of the test environment in this paper is mostly 11 m*11 m. The robot's navigation trajectory is

shown in the Fig. 11(b). Similarly, the robot can reach the target position without any collision.

In Fig. 7, Fig. 9 and Fig. 11, test environments are all static. We have created a 6 m*6m dynamic environment as shown in Fig. 12, showing the process of collision free path plan for robots. Among them, the white square is a dynamic obstacle that moves towards the blue dotted arrow. When it reaches a certain position, it returns its starting position and repeats this process. The blue square is still the target position. Green line is a robot movement trajectory. From Fig. 12, it can be seen that when the white square moves down, the robot bypasses the white obstacle of the movement and successfully achieves the target position. It can be seen that our algorithms also have good ability to avoid dynamic obstacles.

4.3. Performance analysis for LGE-SAC algorithm

To further demonstrate the superiority of LGE-SAC algorithm in this paper, we compare its performance in the test environment shown in Fig. 9(a), 11(a) and Fig. 12 with the SAC (Haarnoja et al., 2018) and

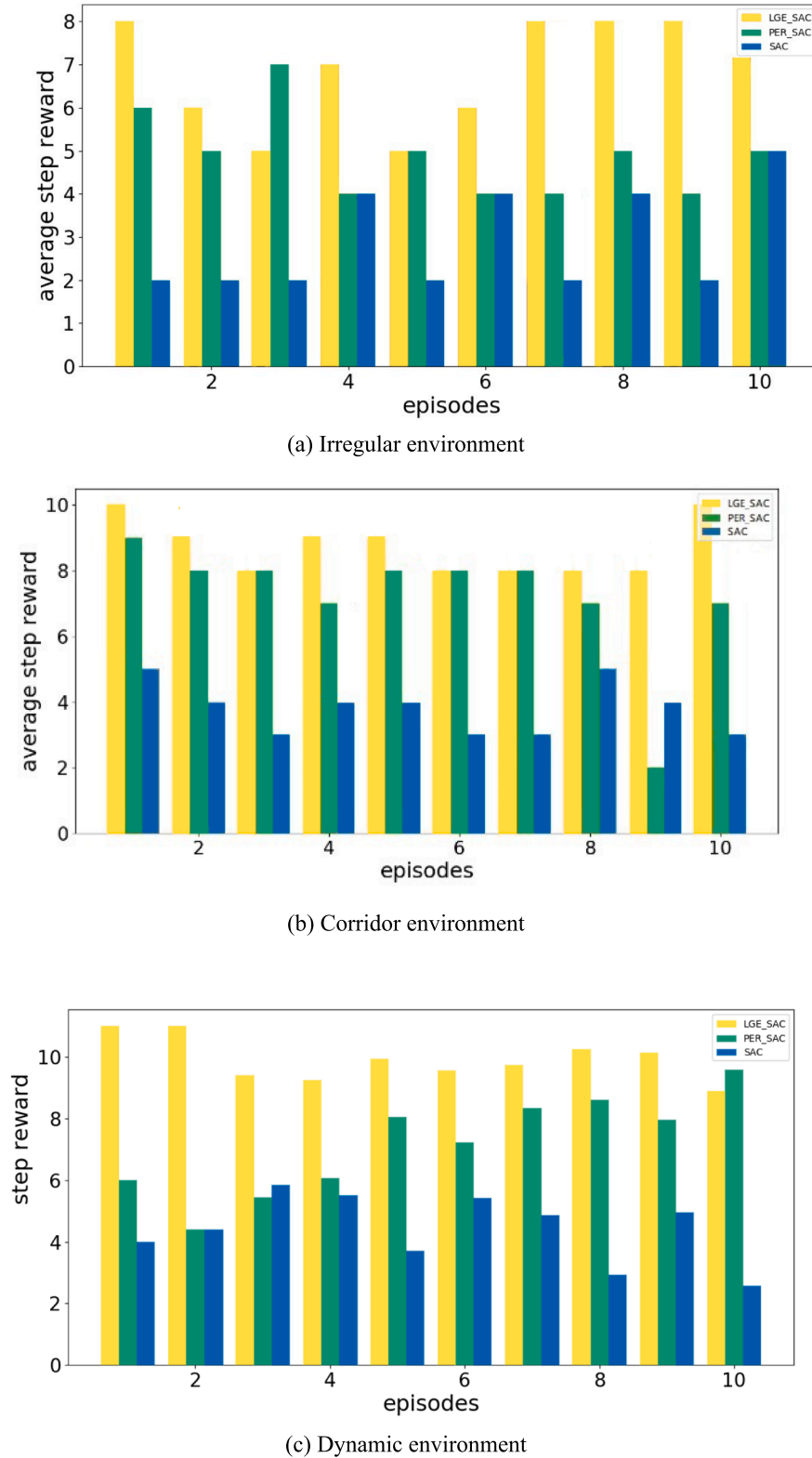


Fig. 13. Average instantaneous reward value of each episode in different environments.

PER-SAC (Lin et al., 2018) algorithms, and analyze from different perspectives of three performance indicators: success rate, instantaneous reward, and path planning time.

Instantaneous reward: We randomly selected 10 episodes from 100 test episodes and subsequently calculate the average instantaneous reward for each of these 10 episodes, that is, the ratio of the total reward

value of the episode to the number of steps planned for each episode, as shown in Fig. 13. Both LGE-SAC and PER-SAC based on prioritized experience replay can increase the instantaneous reward value, but our method can keep the reward value at a relatively high level with small fluctuations. It can be seen that LGE-SAC can keep the robot's planning in a more stable state, and the robustness is improved.

Table 2

Success rate and high reward rate comparison.

Algorithms	Success rate			High reward rate		
	Irregular	Corridor	Dynamic	Irregular	Corridor	Dynamic
SAC	70%	92%	57%	71.42%	29.34%	35%
PER-SAC	56%	84%	43%	58.82%	42.86%	68.2%
LGE-SAC	82%	94%	70%	87.80%	92.55%	90%

Success rate: No matter what environment the robot can successfully reach the target position is an important indicator to measure the performance of autonomous navigation. We test 100 episodes to find the percentage of the total episodes in which the robot can reach the target position without collision. We also calculate the ratio of the number of episodes the robot reached the target point with a high reward value to the number of episodes that the robot successfully reached the target position, as shown in Table 2. The high reward value here refers to the cumulative reward value larger than a certain fixed value. To measure the advantages and disadvantages of strengthening learning strategies, cumulative rewards are generally used as standards. Therefore, the comparison of the algorithm is highlighted by the total number of successful episodes to the number of episodes to reach the target point with high award value. In the three environments, the success rate of the LGE-SAC algorithm is higher than SAC and PER-SAC algorithms. Although the success rate in some environments is close to SAC algorithm, its high reward value rate is much greater than SAC. Although PER-SAC shows an average reward close to LGE-SAC in three episodes in Fig. 13(b), this is merely an occasional performance. When considering the total number of episodes, the LGE-SAC algorithm outperforms PER-SAC. It shows that the LGE-SAC algorithm can steadily reach the target position with a higher reward value by using good past experience without collision.

Path planning time: Path planning time is also an important indicator to measure autonomous navigation capabilities. We record the number of planned steps for 100 episodes, as shown in Fig. 14. Compared with SAC and PER-SAC algorithms, the algorithm proposed in this paper takes less planning time and has a smaller change range. It can be seen that the LGE-SAC algorithm can increase the speed, so that the robot can reach the target point quickly and accurately in an unknown environment.

Randomly select 50 successful turns to reach the target point from the trained model in the irregular environment in Fig. 9, and draw Fig. 15. Compared with the PPO algorithm, the SAC algorithm and LGE-SAC algorithm have fewer planning steps, basically within 200 steps. However, the PPO algorithm has unstable planning steps, which wastes the robot's time. Therefore, the algorithm proposed in this article has higher planning efficiency. The step count range of the SAC algorithm is between 100 and 200, while the LGE-SAC algorithm requires fewer planning steps, stabilizes below 100 steps, and has a small range of changes. Therefore, it can be concluded that the improved LGE-SAC algorithm can improve the planning speed of robots and enable them to quickly and accurately reach the target point.

4.4. RSAC performance

The LGE-SAC algorithm based on prioritized experience replay enables the robot to autonomously plan a collision-free path to reach the target position, and the data utilization rate and algorithm convergence speed are both improved. However, in an environment with a large environmental range (long sequence navigation environment), the space to be explored increases, and it is difficult for the robot to explore the target point. At the same time, deep reinforcement learning generally has the problem of poor generalization performance. When the robot enters a new and unknown environment, it is difficult to plan a collision-free path to the target position by loading the existing over-fitting model. Therefore, aiming at the problems that the robot explores slowly in the long sequence environment, it is difficult to reach the target point, and the generalization performance of the LGE-SAC

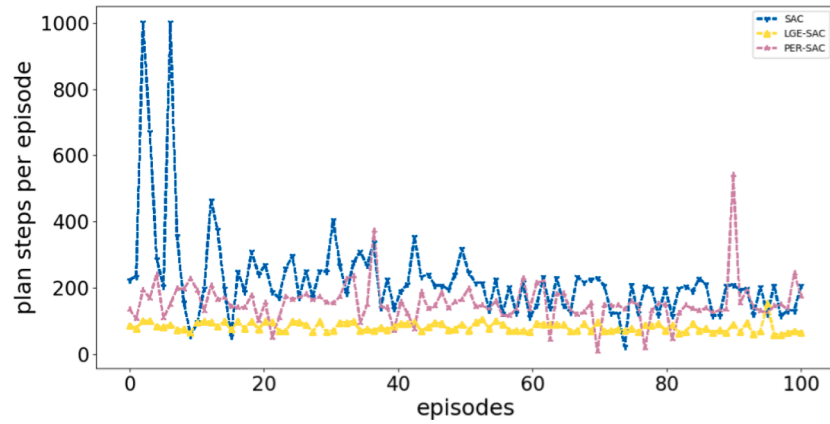
algorithm is poor, this section proposes a deep residual reinforcement learning method based on curiosity-driven exploration for path planning. The simulation experiments and results show that after adding curiosity exploration, the convergence speed of the RSAC algorithm is significantly improved, and it is more suitable for the exploration of long sequence environments by comparing with the LGE-SAC algorithm. Moreover, the introduction of traditional Artificial Potential Field Method enhances the adaptability of the robot to different environments.

First, the algorithm model is trained in the long sequence environment of 10 m*10 m as shown in Fig. 16(a), and after a period of time, the robot can avoid obstacles autonomously and navigate to the target location successfully, and its motion trajectory is shown as the green curve in Fig. 16(b).

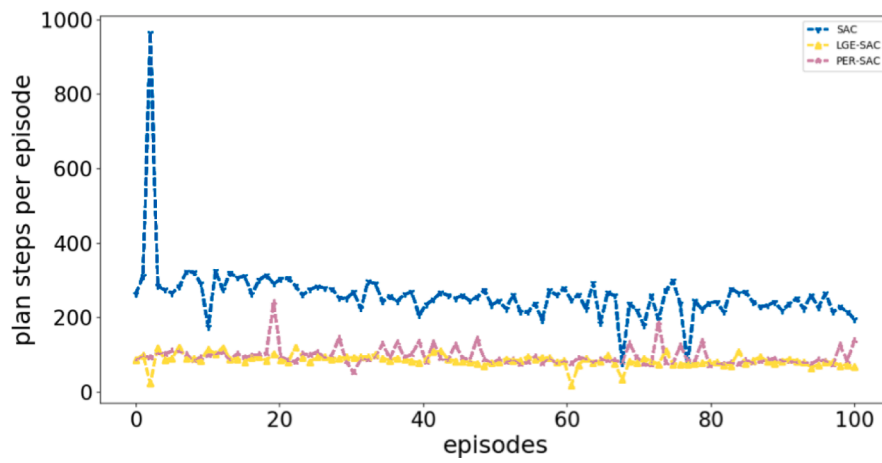
We plot the mean change curve of the cumulative reward value obtained by the robot every 10 episodes as the number of algorithm iteration steps increases, and RSAC is compared and analyzed with SAC algorithm, LGE-SAC algorithm, and classical reinforcement learning algorithm PPO, as shown in Fig. 17. The reward value change curve of LGE-SAC and RSAC algorithms show that they are convergence. The red curve in the figure shows the RSAC algorithm, and the robot can also explore to reach the target location as soon as possible in the long sequence environment because of the introduction of the Artificial Potential Field method as a prior controller. The LGE-SAC algorithm can converge at about 40,000 steps, which is faster than the PPO algorithm and SAC algorithm. Compared with the other three algorithms, RSAC has a very fast convergence speed and solves the problem of long exploration time in the long sequence environment.

To demonstrate the effect of incorporating an attention mechanism and curiosity-driven exploration, we trained RSAC, RSAC without an attention mechanism (RSAC without Attention), and RSAC without curiosity-driven exploration (RSAC without Curiosity) under the same settings in the environment depicted in Fig. 16. The results are presented in the subsequent figures. Fig. 18(a) shows the number of steps per episode, while Fig. 18(b) depicts the cumulative reward per episode. As observed in Fig. 18(a), RSAC with curiosity-driven intrinsic exploration based on an attention mechanism requires fewer steps after training, indicating improved planning efficiency. Furthermore, due to the enhanced efficiency from curiosity-driven exploration and the attention mechanism, the number of steps is reduced even before convergence, resulting in faster convergence. As shown in Fig. 18(b), the convergence of the reward values indicates that RSAC with curiosity-driven intrinsic exploration based on an attention mechanism achieves higher exploration efficiency. Meanwhile, the exploration of more suitable paths reduces the possibility of falling into local optima due to the force balance of the Artificial Potential Field method, leading to more stable rewards and step counts for RSAC. In contrast, without curiosity-driven intrinsic rewards, the algorithm may tend to move directly towards the goal, resulting in insufficient exploration in some cases, thereby reducing training efficiency, increasing the number of steps during stability, and potentially increasing the likelihood of collisions.

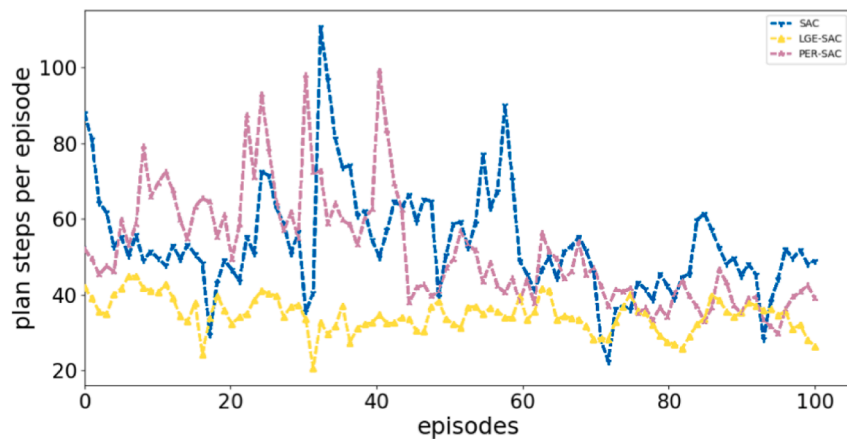
The trained model has already the ability to plan autonomously in the long sequence environment, and it is introduced into the new dynamic environment to continue the training, and its motion process is shown in Fig. 19. Fig. 19 is a relatively dense environment that includes not only narrow aisles, but also obstacles on the path of the robot. The size of the environment is 10 m*10 m. The starting position of the robot



(a) Irregular environment



(b) Corridor environment



(c) Dynamic environment

Fig. 14. Planning steps of each episode in different environments.

is located in a narrow channel, and the white rectangular obstacle makes a round-trip motion along the red dashed line direction, and the target position of the robot is set behind the dynamic obstacle, which increases the difficulty of reaching the target point. As observed from the figure,

the robot can reach the entrance of the narrow passage without collision, where it encounters the dynamic obstacle and successfully avoids it, and continues to move forward. After that, the dynamic obstacle returns to the initial position and blocks the robot's travel path again, but the robot

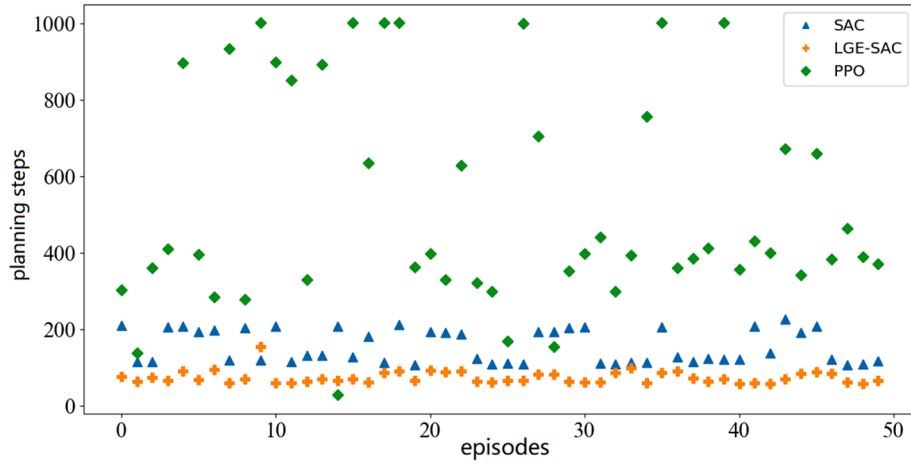


Fig. 15. Comparison of robot planning steps.

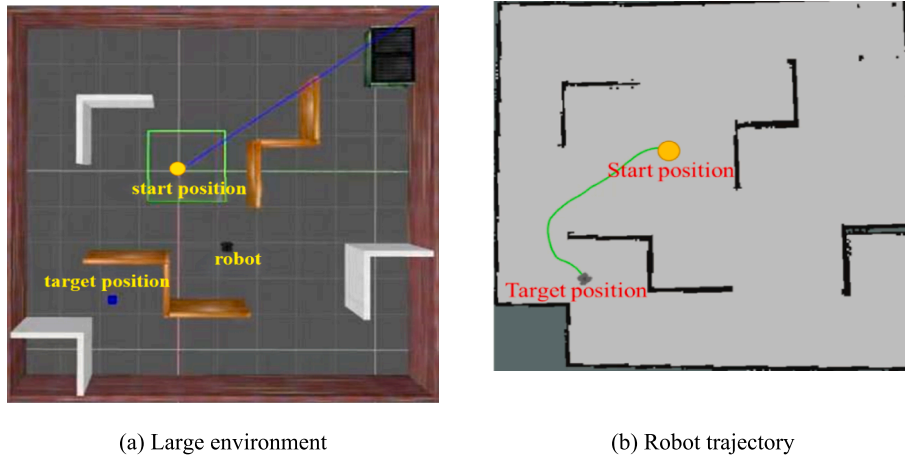


Fig. 16. Large environment and robot trajectory.

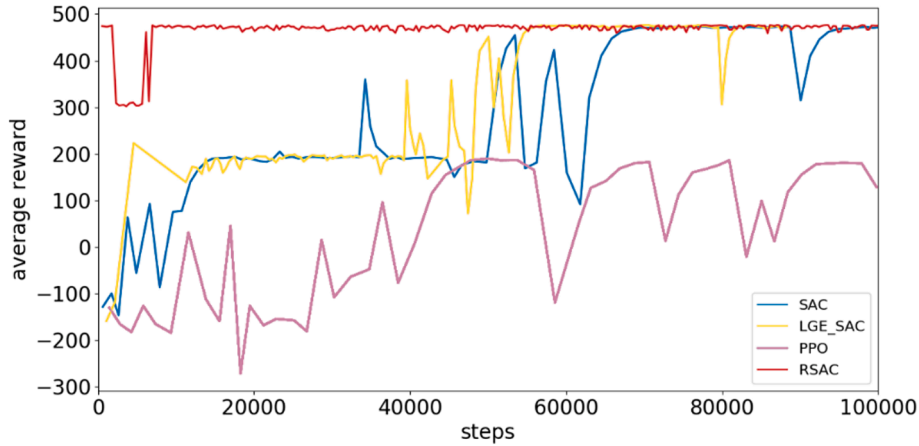


Fig. 17. Average reward.

can still plan accordingly to avoid the obstacle and reach the target location.

To demonstrate the suitability of the RSAC algorithm for path planning in dynamic environments, a variation curve of the average cumulative reward values was obtained by the robot every 10 episodes. The increase in the number of algorithm iterations is plotted in Fig. 20. The disparities in step intervals reflect the varying iteration counts

required by each algorithm to complete the same number of episodes. The PPO algorithm exhibits fewer turning points, attributable to the high number of steps per 10 episodes, indicative of a longer average exploration time per round, and the relatively low reward values suggest that it still struggles to reach the target position. In contrast, SAC, LGE-SAC, and RSAC require fewer steps per 10 episodes. This is because the SAC algorithm incorporates an entropy term into its optimization

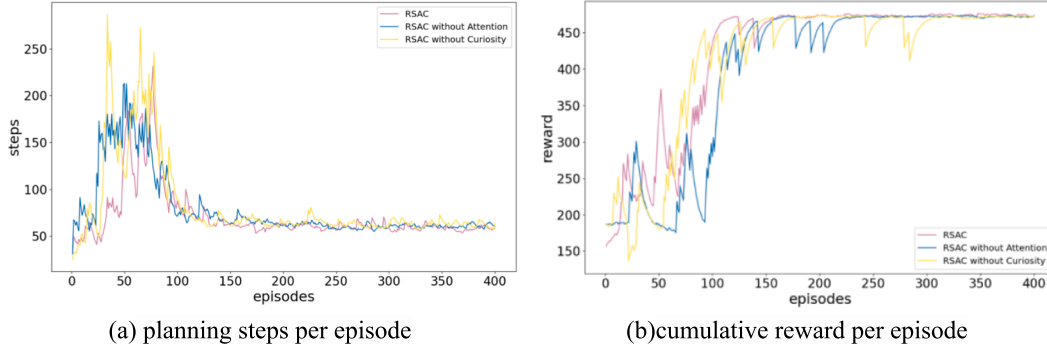


Fig. 18. Comparison of RSAC with and without Attention Mechanism and Curiosity-driven Exploration.

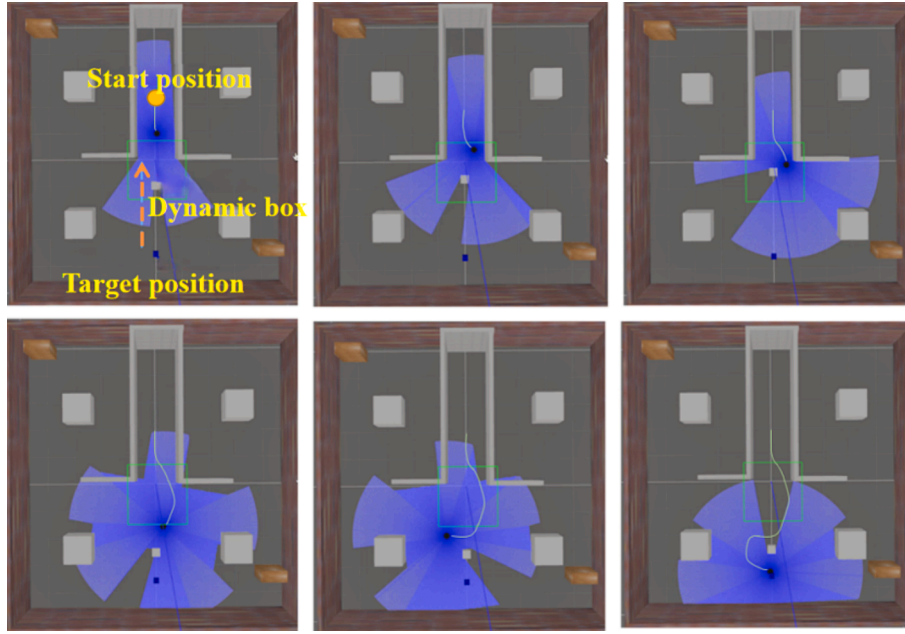


Fig. 19. Path planning process in dynamic environment.

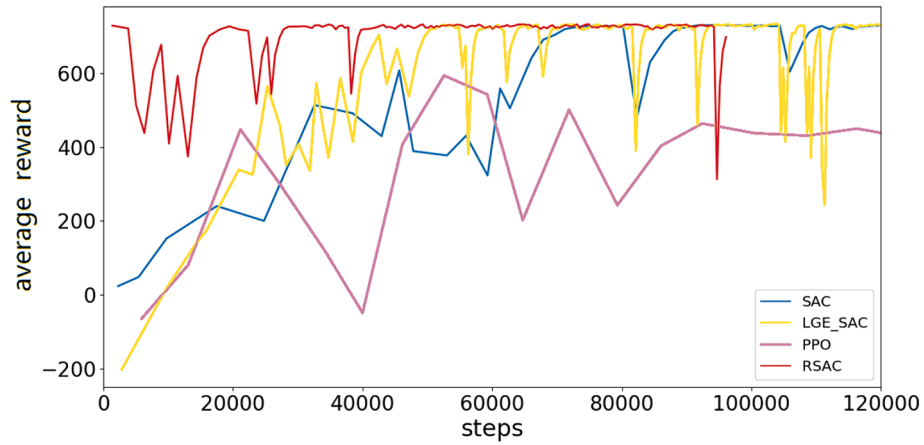


Fig. 20. Average reward.

objective, enhancing randomness and exploration capabilities, which facilitate the exploration of diverse paths during training with dynamic obstacles, potentially leading to earlier episode terminations (i.e., fewer steps per 10 episodes). This, in turn, accelerates the learning of optimal

routes to the target point. In long-sequence environments, RSAC demonstrates fewer steps per 10 episodes with higher reward values, and upon transitioning to a new environment, maintains stable reward values after brief training, underscoring its robust exploration and

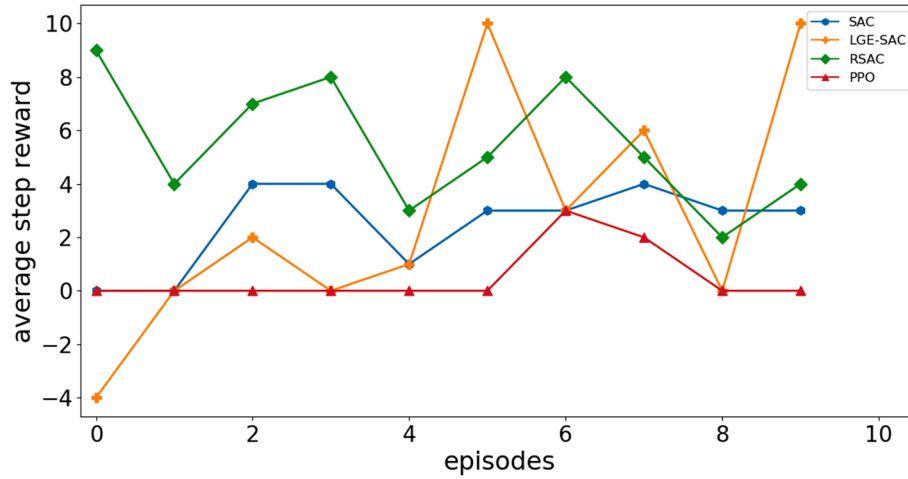


Fig. 21. Average step reward.

obstacle avoidance capabilities, enabling it to more readily reach the target position and learn the shortest path. From the figure, we can see that the RSAC algorithm has the fastest convergence speed, followed by the LGE-SAC algorithm, SAC, and PPO. Therefore, it can be concluded that the RSAC algorithm is more suitable for planning in dynamic environments.

With the guarantee of high episode reward values, it was hoped that the algorithm could also improve in single-step performance, so data from 10 randomly selected episodes are plotted for the variation of instantaneous reward values. This is shown in Fig. 21. Among them, RSAC algorithm has an average instantaneous reward value around 5, which is higher than the other curves. PPO algorithm has the lowest reward value, and the instantaneous reward value is around 0 due to the long convergence time of the PPO algorithm, which is prone to collision. Therefore, it can be seen that the RSAC algorithm is also higher than the other algorithms in terms of single-step performance and has good planning efficiency.

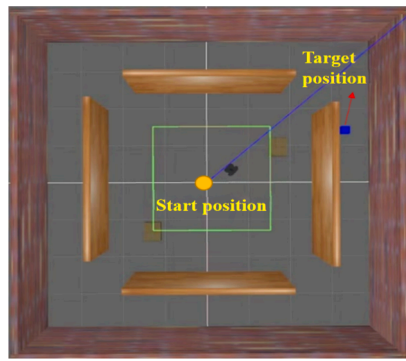
During the robot's path planning, the action selector makes decisions based on the specific conditions of the environment and the operational stage of the algorithm. In the early general state, it is mostly a priori action. Since the RSAC algorithm is to use the traditional Artificial Potential Field method for path planning in the early stage, it accumulates better experience values. In environments with dynamic obstacles or in previously unencountered scenarios, there is a tendency to prefer deep reinforcement learning strategies. Although the empirical value in the early stage is high, traditional path planning methods are usually deterministic, and there are problems of easy oscillation, falling into local optimum and low efficiency of path planning. Therefore, in the later stages or under complex environments, the hybrid approach becomes the primary path planning strategy, integrating the advantages of both traditional methods and deep reinforcement learning to overcome their respective limitations. In the test environment, the rough statistical data of selective operations are shown in Fig. 24.

Autonomous path planning in an unknown complex environment is a challenge for the robot. The RSAC algorithm performance is tested in a static environment as shown in Fig. 22(a), 22(c) and 22(e) to see if the robot can successfully reach the target point. The size of the environment is 8 m*8m, 10 m*10 m and 10 m*10 m. The green curves in Fig. 22 (b), 22(d) and 22(f) show the robot's motion trajectory. As can be seen from the figures, the robot can avoid square obstacles as well as L-shaped obstacles and successfully reach the target location with autonomous planning capability in an unknown static environment.

In order to test the performance of the proposed RSAC algorithm in a new dynamic environment, Fig. 23 shows the movement process of the robot near the initial position of the environment. The size of the

environment is 6 m*6m. The white cylinder moves along the direction of the blue dotted line. When it reaches the target position set by the dynamic obstacle, it returns to the initial position and repeats this movement process. The blue square is the target position of the robot. The green line is a robot movement trajectory. Fig. 23 demonstrates that the robot has the ability to avoid dynamic obstacles. Under the guidance of the prior controller, it can confirm the direction of the target position and move towards it to reach the position of the first baffle corner. Next, we observe whether the robot can pass the corner smoothly, and output the action selection mode of the robot in the test environment, as shown in Fig. 24. Most of the time, the robot's outputs are mixed actions for planning and decision-making. When encountering uncertain situations such as corners that are prone to collisions, the output action of the prior controller is adopted to ensure that the robot will not collide. When the robot passes the corner position, a long baffle appears in front of it to prevent it from reaching the target position. At this moment, the robot tends to fall into a local optimal state and wander around here to explore. Fig. 24 (b) is the action output of the robot near the baffle. The robot will frequently choose the action decision of the Artificial Potential Field method for planning, so as to avoid falling into an endless loop.

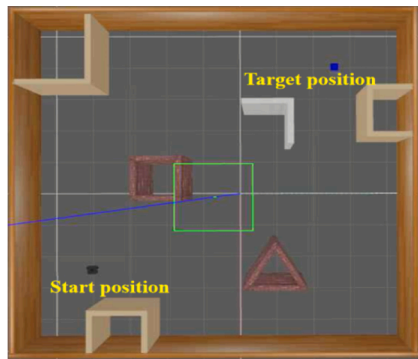
To demonstrate the superiority of the deep residual reinforcement learning framework combining reinforcement learning methods with traditional path planning algorithm APF, we use ablation experiments for validation. The algorithm model trained in the dynamic environment shown in Fig. 19 is migrated to the static environment shown in Fig. 22 (a) and the dynamic environment shown in Fig. 23 for performance testing and comparison with APF method for a total of 50 episodes, and the ratio of the number of episodes that successfully reach the target point to the total number of episodes tested is calculated, as well as the average number of planning steps per robot episode. The results of the proposed algorithm with APF algorithm are shown in Table 3. We evaluate generalizability in terms of the algorithm's success rate, number of planning steps, and stability in a new test environment. By calculating the ratio of the number of episodes successfully reaching the target point to the total number of test episodes, we found that the RSAC algorithm significantly improved in success rate compared to the APF method. This directly reflects the stronger adaptability of the RSAC algorithm in novel environments. Additionally, we compared the average number of planning steps per episode for the robot, and the results showed that the RSAC algorithm was more efficient in path planning. Furthermore, in static test environments, we observed that the Artificial Potential Field method would always fall into local optimum and fail to reach the target location. In the dynamic test environment, the success rate of the RSAC algorithm is lower than that in the static environment, but still higher than that of the Artificial Potential Field method. The



(a) Static environment



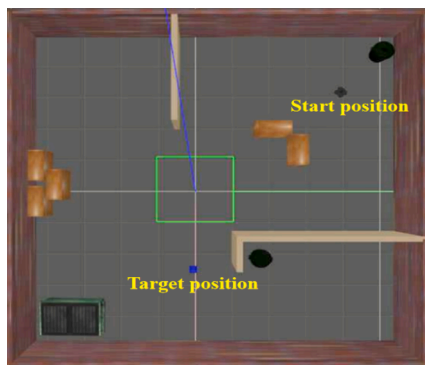
(b) Trajectory in the static environment



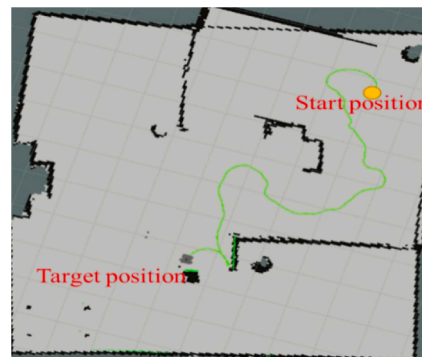
(c) Static environment



(d) Trajectory in the static environment



(e) Static environment



(f) Trajectory in the static environment

Fig. 22. Static test environment in Gazebo and trajectory in Rviz.

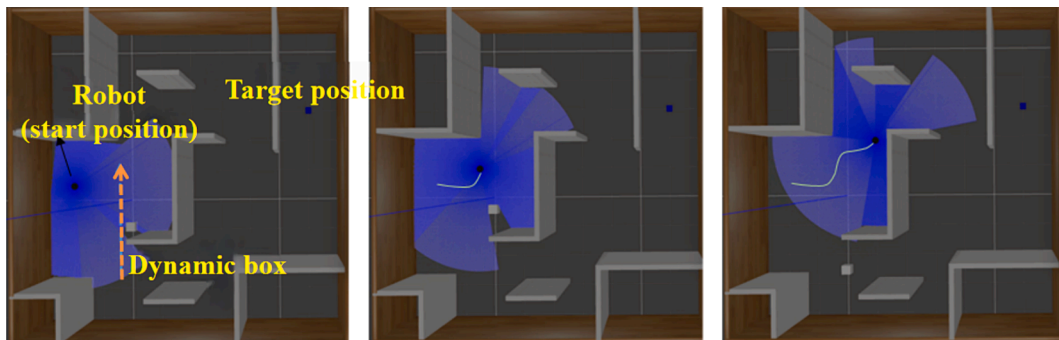


Fig. 23. The process of avoiding dynamic disorders from robots.

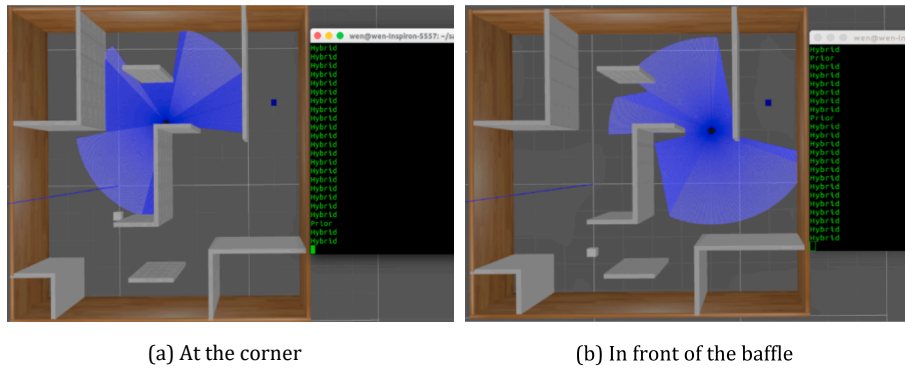


Fig. 24. Movement selection of robots in different positions.

Table 3

Success rate and high reward rate comparison of the proposed algorithm RSAC and the traditional algorithm APF.

Algorithms	Success rate		Plan steps	
	RSAC	APF	RSAC	APF
Static environment	76%	40%	480	822
Dynamic environment	70%	2%	268	900

Table 4

Algorithm Computational Efficiency Table.

Algorithm	Iteration duration (ms)	Time steps to solve	Time to solve (s)
RSAC	65.2	8,000	4,080
LGE-SAC	56.7	40,000	15,000
SAC	36.4	75,000	38,000

RSAC algorithm maintains a high success rate and a low number of planning steps in both static and dynamic test environments, demonstrating the stability of its performance. Therefore, in general, the RSAC algorithm makes up for the disadvantages of traditional path planning methods and improves the autonomous adaptation capability to new environments.

We tallied the planned time steps and the actual runtime required for convergence of the cumulative reward during the training of our

algorithm in the environment depicted in Fig. 16, as shown in Table 4. In this context, “Iteration duration” signifies the time required for a single iteration of the algorithm training, “Time steps to solve” denotes the number of steps it takes for the agent to interact with the environment until convergence is achieved, and “Time to solve (seconds)” represents the total time required for the agent to complete convergence in the simulation environment, encompassing processes such as algorithm iteration, data collection, and policy updates. Consequently, a single training iteration of RSAC necessitates approximately 65 ms, and the model training necessitates a total of 4080 s (8000 iterations).

4.5. Experiments in real scenarios

The hardware device used in this experiment is NVIDIA GeForce GTX 1080 GPU and Intel Core i7 CPU (2.9 GHZ). Turtlebot3 Waffle Pi with a Raspberry Pi board is selected as the robot model, and the simulation environment is built in the Gazebo emulator in the robot operating system (Robot Operating System, ROS). TurtleBot3 Waffle Pi is a platform based on the two-wheel differential drive type, equipped with a 360° LiDAR. The ranging range of the LiDAR is 0.12 m to 3.5 m, that is, the distance between the obstacles scanned in the environment and the robot. During the experimental process, if the data obtained by scanning the surrounding environment is greater than 3.5 m, then the robot is 3.5 m away from obstacles in this direction. The software and hardware of TurtleBot3 Waffle Pi are open source and suitable for ROS communication.

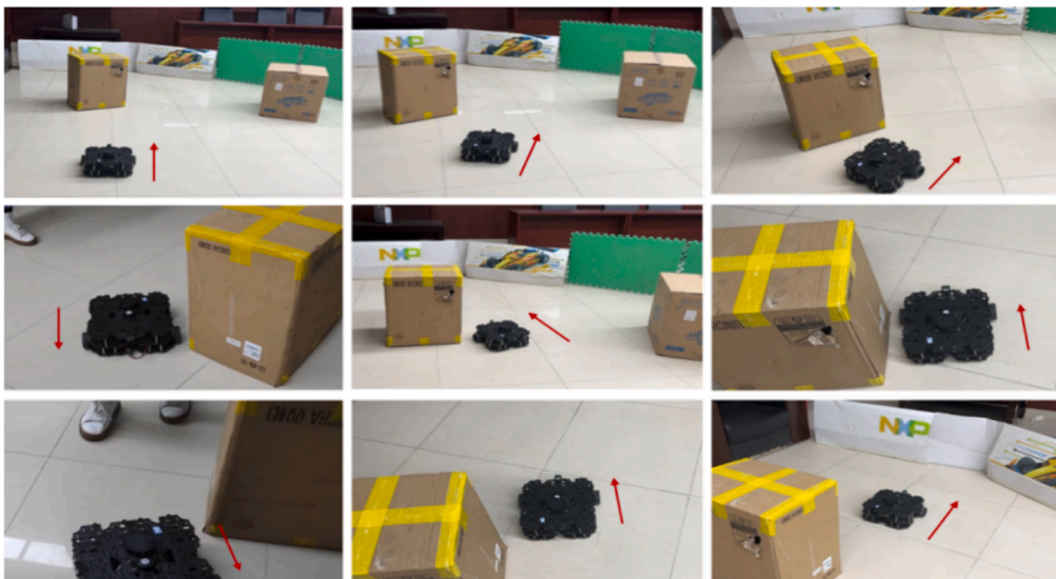


Fig. 25. The robot motion process in scene 1 (static obstacles).



Fig. 26. The robot motion trajectory in scene 1.



Fig. 27. The robot motion process in scene 2(static obstacles and dynamic obstacles).

In this section, the RSAC algorithm is experimentally validated by migrating the deep residual reinforcement learning algorithm model, which is completed by training in a simulation environment, to a real Turtlebot3. Two different experiment environments are created in this section, including scenario one containing static obstacles and scenario two containing static and dynamic obstacles.

Firstly, we create scene1 shown in Fig. 25, which includes two cartons. We take pictures of the robot movement process from multiple angles, and the red arrows show the direction of robot movement. As can be seen in Fig. 25, after encountering the carton directly in front of it, the robot starts to turn from its original straight direction and passes through the channel smoothly to reach the target position. The robot can build the map by using the Gmapping algorithm while navigation, as shown in Fig. 26. It can be concluded that the RSAC algorithm has good obstacle avoidance ability as well as the ability of autonomous path planning under real static scenes.

After that, the performance of the algorithm is verified in Scene 2, which contains static and dynamic obstacles, as shown in Fig. 27, where the human walks along the orange dashed part and the robot's motion direction is indicated by the red arrow. Similarly, the robot can walk while building the environment map according to the Gmapping algorithm, and the green curve in Fig. 28 is the robot's motion trajectory. The experimental results show that the RSAC algorithm also has good autonomous planning capability in dynamic environments.

5. Conclusion

This paper proposes the RSAC algorithm for the autonomous navigation and obstacle avoidance process of robots. Our algorithm promotes the exploitation of experience data through LGE-SAC algorithm, a method based on prioritized replay of good experiences, and uses attention-based curiosity-driven method to encourage robot



Fig. 28. The robot motion trajectory in scene 2.

exploration. Most importantly, we fuse the reinforcement learning policy with the Artificial Potential Field method, which improves the generalization performance of the algorithm. We compared our algorithm with SAC, PPO, PER-SAC based on common prioritized experience replay. The experimental results show that the proposed algorithm increases the instantaneous reward value and reduces the path planning time, and the success rate of path planning is also greater than other algorithms. Also, the success rate of navigation in unknown environments is improved. It can be seen that imitating good past experience and combining traditional method can improve the autonomous navigation ability of the robot.

In our future work, we will adopt a decaying learning rate strategy and introduce *meta-learning* and transfer learning methods to further enhance the generalization capability of our model from the perspective of policy updates.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Shuhuan Wen reports financial support was provided by National Natural Science Foundation of China.

Data availability

Data will be made available on request.

Acknowledgments

The work was partly supported by the National Natural Science Foundation of China (NSFC, Project No. 62273296), the National Natural Science Foundation of China and the Royal Society of Britain (NSFC-RS, Project No. 62111530148) and the China Scholarship Council (CSC, Project No.201908130016)

References

- Ahmed, A., Mohammad, Y.F., Parque, V., El-Hussieny, H., & Ahmed, S.M. (2022). End-to-End Mobile Robot Navigation using a Residual Deep Reinforcement Learning in Dynamic Human Environments. *2022 18th IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA)*, 1-6.
- Bach, N., Melnik, A., Schilling, M., Korthals, T., & Ritter, H. (2020). Learn to Move Through a Combination of Policy Gradient Algorithms: DDPG, D4PG, and TD3. In

- Machine Learning, Optimization, and Data Science* (pp. 631–644). doi:10.1007/978-3-030-64580-9_52.
- Carroll, M., Liu, Z., Kasaei, M. M., & Li, Z. (2023). Agile and versatile robot locomotion via kernel-based residual learning. *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, 5148–5154.
- Chalmers, E., Contreras, E. B., Robertson, B., Luczak, A., & Gruber, A. (2018). Learning to predict consequences as a method of knowledge transfer in reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, 29(6), 2259–2270. <https://doi.org/10.1109/TNNLS.2017.2690910>
- Du, H., Yu, X., & Zheng, L. (2020). Learning object relation graph and tentative policy for visual navigation. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part VII 16* (pp. 19–34). Springer International Publishing.
- Ejaz, M. M., Tang, T. B., & Lu, C.-K. (2021). Vision-based autonomous navigation approach for a tracked robot using deep reinforcement learning. *IEEE Sensors Journal*, 21(2), 2230–2240. <https://doi.org/10.1109/JSEN.2020.3016299>
- Fang, K., Toshev, A., Fei-Fei, L., & Savarese, S. (2019). Scene memory transformer for embodied agents in long-horizon tasks. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, 538–547.
- Frans, K., & Isola, P. (2022). Powderworld: A platform for understanding generalization via rich task distributions. *ArXiv, abs/2211.13051*.
- Fu, J., Co-Reyes, J., & Levine, S. (2017). Ex2: Exploration with exemplar models for deep reinforcement learning. *Advances in neural information processing systems*, 30.
- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning* (pp. 1861–1870). PMLR.
- Henriques, J. F., & Vedaldi, A. (2018). MapNet: An allocentric spatial memory for mapping environments. *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, 8476–8484. <https://doi.org/10.1109/CVPR.2018.00884>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9, 1735–1780.
- Huang, B., Zhao, T., Yue, M., & Wang, J. (2024). Bi-level adaptive storage expansion strategy for microgrids using deep reinforcement learning. *IEEE Transactions on Smart Grid*, 15, 1362–1375.
- Iscen, A., Caluwaerts, K., Tan, J., Zhang, T., Coumans, E., Sindhwani, V., & Vanhoucke, V. (2018). Policies modulating trajectory generators. In *Conference on Robot Learning* (pp. 916–926). PMLR.
- Lai, X., Chen, Y., Lu, F., Liu, J., & Jia, J. (2023). Spherical Transformer for LiDAR-Based 3D Recognition. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 17545–17555.
- Lashkari, N., Biglarbegian, M., & Yang, S. X. (2020). Development of a novel robust control method for formation of heterogeneous multiple mobile robots with autonomous docking capability. *IEEE Transactions on Automation Science and Engineering*, 17, 1759–1776.
- Li, H., Zhang, Q., & Zhao, D. (2020). Deep reinforcement learning-based automatic exploration for navigation in unknown environment. *IEEE Transactions on Neural Networks and Learning Systems*, 31(6), 2064–2076. <https://doi.org/10.1109/TNNLS.2019.2927869>
- Lin, Z., Zhao, T., Yang, G., & Zhang, L. (2018). Episodic memory deep Q-networks. *International Joint Conference on Artificial Intelligence*.
- Liu, Z., Cao, Y., Chen, J., & Li, J. (2023). A Hierarchical reinforcement learning algorithm based on attention mechanism for UAV autonomous navigation. *IEEE Transactions on Intelligent Transportation Systems*, 24, 13309–13320.
- Lotfi, M., Osório, G. J., Javadi, M. S., Ashraf, A., Zahran, M., Samih, G., & Catalão, J. P. S. (2021). A dijkstra-inspired graph algorithm for fully autonomous tasking in

- industrial applications. *IEEE Transactions on Industry Applications*, 57(5), 5448–5460. <https://doi.org/10.1109/TIA.2021.3091418>
- Lu, X., Lee, K., Abbeel, P., & Tiomkin, S. (2020). Dynamics Generalization via Information Bottleneck in Deep Reinforcement Learning. *ArXiv, abs/2008.00614*.
- Ma, F., Liu, G., & Zhang, K. (2021). Goal-conditioned Behavioral Cloning with Prioritized Sampling. 2021 *IEEE International Conference on Networking, Sensing and Control (ICNSC)*, 1, 1–6.
- Malik, D., Li, Y., & Ravikumar, P. (2021). When is generalizable reinforcement learning tractable? *Advances in Neural Information Processing Systems*, 34, 8032–8045.
- Mingo Hoffman, E., Traversaro, S., Rocchi, A., Ferrati, M., Settimi, A., Romano, F., & Tsarakis, N. (2014). Yarp based plugins for gazebo simulator. *Modelling and Simulation for Autonomous Systems*, 8906, 333–346. https://doi.org/10.1007/978-3-319-13823-7_29
- Moos, S., Lee, J., & Song, H. O. (2022). Rethinking value function learning for generalization in reinforcement learning. *Advances in Neural Information Processing Systems*, 35, 34846–34858.
- Oh, J., Guo, Y., Singh, S., & Lee, H. (2018). Self-imitation learning. In *International Conference on Machine Learning* (pp. 3878–3887). PMLR.
- Osband, I., Blundell, C., Pritzel, A., & Van Roy, B. (2016). Deep exploration via bootstrapped DQN. *Advances in neural information processing systems*, 29.
- Pathak, D., Agrawal, P., Efron, A. A., & Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017, 488–489. <https://doi.org/10.1109/CVPRW.2017.70>
- Pfeiffer, M., Shukla, S., Turchetta, M., Cadena, C., Krause, A., Siegwart, R., & Nieto, J. (2018). Reinforced imitation: Sample efficient deep reinforcement learning for mapless navigation by leveraging prior demonstrations. *IEEE Robotics and Automation Letters*, 3(4), 4423–4430. <https://doi.org/10.1109/LRA.2018.2869644>
- Rana, K., Dasagi, V., Haviland, J., Talbot, B., Milford, M., & Sünderhauf, N. (2023). Bayesian controller fusion: Leveraging control priors in deep reinforcement learning for robotics. *The International Journal of Robotics Research*, 42(3), 123–146.
- Rana, K., Dasagi, V., Talbot, B., Milford, M., & Sünderhauf, N. (2020). Multiplicative controller fusion: Leveraging algorithmic priors for sample-efficient reinforcement learning and safe sim-to-real transfer. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, 6069–6076. <https://doi.org/10.1109/IROS45743.2020.9341372>
- Rana, K., Talbot, B., Dasagi, V., Milford, M., & Sünderhauf, N. (2020). Residual reactive navigation: Combining classical and learned navigation strategies for deployment in unknown environments. *IEEE International Conference on Robotics and Automation (ICRA)*, 2020, 11493–11499. <https://doi.org/10.1109/ICRA40945.2020.9197386>
- Reizinger, P., & Szemenyei, M. (2020). In *Attention-based curiosity-driven exploration in deep reinforcement learning* (pp. 3542–3546). IEEE.
- Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2015). Prioritized experience replay. *arxiv preprint arxiv:1511.05952*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arxiv preprint arxiv:1707.06347*.
- Shi, H., Shi, L., Xu, M., & Hwang, K.-S. (2020). End-to-end navigation strategy with deep reinforcement learning for mobile robots. *IEEE Transactions on Industrial Informatics*, 16(4), 2393–2402. <https://doi.org/10.1109/TII.2019.2936167>
- Truong, X.-T., & Ngo, T. D. (2017). Toward socially aware robot navigation in dynamic and crowded environments: A proactive social motion model. *IEEE Transactions on Automation Science and Engineering*, 14, 1–18. <https://doi.org/10.1109/TASE.2017.2731371>
- Watkins-Valls, D., Xu, J., Waytowich, N., & Allen, P. (2020). Learning your way without map or compass: Panoramic target driven visual navigation. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, 5816–5823. <https://doi.org/10.1109/IROS45743.2020.9341511>
- Wen, S., Wen, Z., Zhang, D., Zhang, H., & Wang, T. (2021). A multi-robot path-planning algorithm for autonomous navigation using meta-reinforcement learning based on transfer learning. *Applied Soft Computing*, 110, Article 107605.
- Wortsman, M., Ehsani, K., Rastegari, M., Farhadi, A., & Mottaghi, R. (2019). Learning to learn how to learn: Self-adaptive visual navigation using meta-learning. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, 6743–6752. <https://doi.org/10.1109/CVPR.2019.00691>
- Wu, J., Sun, Y., Li, D., Shi, J., Li, X., Gao, L., Yu, L., Han, G., & Wu, J. (2023). An adaptive conversion speed Q-learning algorithm for search and rescue UAV path planning in unknown environments. *IEEE Transactions on Vehicular Technology*, 72, 15391–15404.
- Xia, F., Li, C., Mart'in-Mart'in, R., Litany, O., Toshev, A., & Savarese, S. (2020). RelMoGen: Leveraging Motion Generation in Reinforcement Learning for Mobile Manipulation. *ArXiv, abs/2008.07792*.
- Xiao, X., Liu, B., Warnell, G., & Stone, P. (2022). Motion planning and control for mobile robot navigation using machine learning: A survey. *Autonomous Robots*, 46. <https://doi.org/10.1007/s10514-022-10039-8>
- Xie, X., Li, C., Zhang, C., Zhu, Y., & Zhu, S.-C. (2019). Learning virtual grasp with failed demonstrations via bayesian inverse reinforcement learning. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, 1812–1817. <https://doi.org/10.1109/IROS40897.2019.8968063>
- Xue, Y., & Chen, W. (2024). Combining motion planner and deep reinforcement learning for UAV navigation in unknown environment. *IEEE Robotics and Automation Letters*, 9, 635–642.
- Yang, J., Ni, J., Xi, M., Wen, J., & Li, Y. (2023). Intelligent path planning of underwater robot based on reinforcement learning. *IEEE Transactions on Automation Science and Engineering*, 20, 1983–1996.
- Ye, G., Lin, Q., Juang, T.-H., & Liu, H. (2020). Collision-free navigation of human-centered robots via Markov games. *IEEE International Conference on Robotics and Automation (ICRA)*, 2020, 11338–11344. <https://doi.org/10.1109/ICRA40945.2020.9196810>
- Ye, M., Dong, X., Zhao, J., & Huang, Z. (2022). Path planning of manipulator based on improved RRT* algorithm. *Journal of Physics: Conference Series*, 2365.
- Yoon, H.-S., Lee, S.-H., & Seo, S.-W. (2020). Exploration strategy based on validity of actions in deep reinforcement learning. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, 6134–6139.
- Zhang [Zhen], Z., Wang, D., & Gao, J. (2021). Learning automata-based multiagent reinforcement learning for optimization of cooperative tasks. *IEEE Transactions on Neural Networks and Learning Systems*, 32, 4639–4652.
- Zhang [Zhe], Z., Wu, J., Dai, J., & He, C. (2020). A novel real-time penetration path planning algorithm for stealth UAV in 3D complex dynamic environment. *IEEE Access*, 8, 122757–122771.
- Zhu, Y., Mottaghi, R., Kolve, E., Lim, J. J., Gupta, A., Fei-Fei, L., & Farhadi, A. (2017). Target-driven visual navigation in indoor scenes using deep reinforcement learning. *IEEE International Conference on Robotics and Automation (ICRA)*, 2017, 3357–3364. <https://doi.org/10.1109/ICRA.2017.7989381>